

Identity-Based Encryption With Equality Test and Flexible Time-Based Authorization for Cloud Computing

Cong Li , Xinyu Feng , Qingni Shen , and Zhonghai Wu 

Abstract—Identity-based encryption with equality test (IBEET) extends identity-based encryption to support the comparison between messages in two distinct ciphertexts associated with different identities. Nevertheless, nearly all of the previous IBEET schemes have the problem that once the trapdoor is sent to the cloud server, it becomes uncontrollable. Meanwhile, the only existing IBEET scheme coping with this problem, namely IBEET with datestamp-based authorization (IBEET-DBA), suffers from the following flaws. (1) A trapdoor merely expresses a single time interval, which is inexpressive. (2) It is inflexible due to only supporting limited types of authorization. (3) It is merely proven secure in the random oracle model, which has been pointed out that it may lead to security risks in practicality. Considering the aforementioned problems, we propose an IBEET with flexible time-based authorization (IBEET-FTBA) scheme, supporting wildcards and putting no constraint on authorization types. We prove our scheme secure against adaptive chosen-ciphertext attacks in the standard model. Subsequently, we give a detailed theoretical analysis of our IBEET-FTBA construction and the main existing IBEET ones. Furthermore, we implement ours and the IBEET-DBA construction as a benchmark, and conduct numerous experiments. Both results demonstrate that ours has better performance in the scenario of multiple time intervals.

Index Terms—Identity-based encryption, equality test, time-based authorization, wildcards, standard model, CCA security.

I. INTRODUCTION

CLOUD computing is a promising technology allowing corporations and individuals to store their data in cloud servers and outsource part of computation to them. In view of the cloud server being honest-but-curious and under various attacks launched by insider and outsider attackers, the data owner

tends to outsource the encrypted data. However, the ciphertext of the traditional public key encryption cannot be computed as the calculation of the corresponding plaintext data. So how to perform the computation on the ciphertext is a great challenge for the cryptographic community. To tackle this problem, the homomorphic encryption [1] and searchable encryption [2] were proposed. The former primitive supports to conduct additive or multiplicative calculations on the ciphertext, and the latter one allows to perform the search operation on it. Then in 2010, Yang et al. [3] proposed a new cryptographic primitive, called public key encryption with equality test (PKEET). It supports to check whether two ciphertexts with distinct public keys are encryptions of a same message. Subsequently, numerous studies [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25] are devoted to enhancing PKEET from the aspects of authorization mechanism, functionality, security, etc. PKEET has many valuable real-world applications, such as encrypted email systems [9] and health record systems [22]. Meanwhile, it can be also applied to build other cryptographic primitives like the searchable encryption [3] and private set intersection [26] as the underlying building block.

Ma [27] first combined the concepts of PKEET and identity-based encryption (IBE), and presented a new primitive called identity-based encryption with equality test (IBEET). It simplifies the tedious certificate management in PKEET. Furthermore, she also defined the security model for IBEET, namely, one-wayness against adaptive identity and adaptive chosen-ciphertext attacks (OW-ID-CCA2) security. Nevertheless, Lee et al. [11] pointed out that the security model proposed in [27] is incomplete as it does not consider the attack launched by the adversary who does not possess the trapdoor. Except for OW-ID-CCA2, they further formalized the indistinguishability against adaptive identity and adaptive chosen-ciphertext attacks (IND-ID-CCA2) security. Besides, they put forward a semi-generic method to build the IBEET construction. Afterwards, a series of studies [12], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37] improve IBEET to achieve the filtered property, group authorization mechanism, high efficiency and resistance against the insider attack. Recently, aiming at the problem that once the trapdoor is delivered to the cloud server, the user will completely lose its control and the server can use it to compare the corresponding ciphertexts generated at any time, Lin et al. [38] introduced the time-based authorization mechanism into IBEET and proposed an IBEET with datestamp-based authorization

Received 29 September 2025; revised 26 April 2026; accepted 28 May 2026. Date of publication 10 June 2026; date of current version 1 September 2026. This work was supported in part by Beijing Natural Science Foundation under Grant 4254081 and in part by National Key Research and Development Program of China under Grant 2022YFB2703301. (Corresponding authors: Zhonghai Wu; Qingni Shen.)

Cong Li, Qingni Shen, and Zhonghai Wu are with the School of Software and Microelectronics, Peking University, Beijing 102600, China, also with the National Engineering Research Center for Software Engineering, Peking University, Beijing 100871, China, also with the Beijing Key Laboratory of Data Intelligence and Security, Peking University, Beijing 100871, China, and also with the Key Laboratory of High Confidence Software Technologies of Ministry of Education, Peking University, Beijing 100871, China (e-mail: li.cong@pku.edu.cn; qingnishen@ss.pku.edu.cn; wuzh@pku.edu.cn).

Xinyu Feng is with Zhonghe Tech (Xiong'an) Co., Ltd., Xiong'an 071700, China (e-mail: vontroy@foxmail.com).

Digital Object Identifier 10.1109/TDSC.2026.3702092

(IBEET-DBA) scheme. It provides three types of authorization, namely, year-type, month-type and day-type.¹ Furthermore, they also proved their IBEET-DBA scheme chosen-ciphertext attacks (CCA) secure in the random oracle model.

A. Motivation

Unfortunately, when applied to real-world scenarios, the IBEET-DBA scheme [38] has the following flaws.

- *Single time interval.* In the IBEET-DBA scheme [38], a trapdoor can only express a single continuous time interval (e.g., a whole year/month/day). However, in practice, users may hope to authorize the test server to compare the ciphertexts created in some day of every month in 2024, such as 7th of each month. In this circumstance, they need to generate 12 trapdoors and send all of them to the test server, which incurs heavy redundancy costs in computation, communication and storage.
- *Limited types of authorization.* It is inflexible and bounded owing to merely supporting three types of authorization. In many applications, for instance, the vehicle system, the vehicle sensor collects the data every second or even every millisecond, so the one-day period (i.e., the minimum period for authorization in the IBEET-DBA scheme [38]) is too coarse-grained to meet the requirement. Besides, in the accounting system, a fiscal year is often not equal to the calendar year,² so the support of self-defined time intervals is required. Apparently, the IBEET-DBA scheme [38] does not have this capability. Overall, it is not suitable for the above systems.
- *Relying on random oracles.* Canetti et al. [39] pointed out that some encryption or signature schemes proven secure in the random oracle model may be insecure in practicality. Goldwasser and Tauman Kalai [40] showed that the analogous problem existing in several identification schemes as well. Moreover, [38] does not account for how to instantiate the random oracle with an appropriate hash function acting as a random oracle to create a construction that can be realized in practice.

Consequently, the aforementioned issues lead us to think about the following question:

Can we design an IBEET with time-based authorization scheme in the standard model, which supports multiple time intervals and puts no restriction on the type of authorization simultaneously?

B. Our Contributions

The contributions of this paper can be summarized as follows.

- 1) We propose an **Identity-Based Encryption with Equality Test and Flexible Time-Based Authorization** scheme, dubbed IBEET-FTBA. According to Table I, our IBEET-FTBA scheme is the first IBEET scheme that achieves

¹Year-type/month-type/day-type denotes that the trapdoor can be utilized for testing ciphertexts generated in some specified year/month/day, such as 2024/March 2024/March 1, 2024.

²The fiscal year of U.S. federal government runs from October 1 to September 30 next year.

TABLE I
COMPARISON BETWEEN OURS AND THE MAIN EXISTING IBEET SCHEMES

Scheme	Security	Model	TBA	NoAT	Express. of trap.
[27]	OW-ID-CCA2	ROM	×	—	—
[11]	OW/IND-ID-CCA2	ROM	×	—	—
[31]	W-IND-ID-CCA*	ROM	×	—	—
[38]	OW/IND-ID-CCA2	ROM	✓	Three	Year/Month/Day
Ours	OW/IND-ID-CCA2	ST	✓	l	Wildcard

OW/IND-ID-CCA2: OW-ID-CCA2 and IND-ID-CCA2, W-IND-ID-CCA2: weak IND-ID-CCA2, *: existing an attack on this security notion, ROM: random oracle model, ST: standard model, TBA: time-based authorization, NoAT: maximum number of authorization types, l : maximum depth of time interval tree (not including the root node), Express. of trap.: expressiveness of a trapdoor.

the time-based authorization and does not rely on random oracles simultaneously.

- *Multiple time intervals.* By embedding wildcards into the trapdoor, our scheme is expressive and supports multiple time intervals. As shown in Fig. 1, when authorizing the test server to test the ciphertexts generated in 7th of every month in 2024, the user just needs to generate a trapdoor related with the pattern vector $P = (2024, *, 7)$ in our system instead of creating 12 trapdoors like in the IBEET-DBA system [38].
 - *Flexible authorization.* Compared with the IBEET-DBA scheme [38], our scheme has a more flexible authorization mechanism, which no longer limits the number of authorization types to three and can support more kinds of time periods, such as minute, second, millisecond or any self-defined one.
 - *Without random oracles.* The security model for our IBEET-FTBA is firstly formalized. Then our scheme is proven OW-ID-CCA2 secure against Type-I adversaries possessing trapdoors for executing equality test operations and IND-ID-CCA2 secure against Type-II adversaries not possessing trapdoors in the standard model without employing any additional cryptographic primitive (e.g., one-time strong signature, message authentication code, etc.).
- 2) We give detailed performance analyses for our scheme and the main existing IBEET schemes in the theoretical and experimental aspects. Both results show that our scheme has better performance in the situation of multiple time intervals. Furthermore, the advantage becomes more apparent with the growth in the number of time intervals.
 - 3) We further present an extension of our IBEET-FTBA scheme. It has a better expression capability, which supports more complex situations of multiple time intervals, for instance, multiple cross-year time intervals (from July 2022 to January 2023 and from September 2023 to February 2024).

C. Organization

The rest of this paper primarily introduces our IBEET-FTBA from the aspects of the concrete construction, security analysis, performance analysis and discussion. Section II summarizes key literature on PKEET, IBEET and

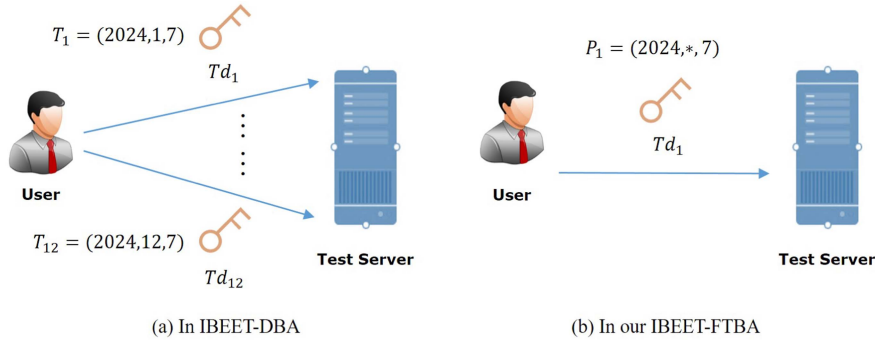


Fig. 1. An example of the authorization process in the scenario of multiple time intervals.

ABEET, respectively. Section III illustrates the notations employed in our IBEET-FTBA, bilinear maps and the truncated decision q -ABDHE assumption. Section IV describes the system architecture, definition and security model of IBEET-FTBA. Section V presents a concrete IBEET-FTBA construction and a formal security analysis of it. Section VI provides the theoretical and experimental analyses of our construction and the main existing IBEET constructions in detail. Section VII discusses how to extend our IBEET-FTBA to support more situations of multiple time periods. Section VIII ends up with a brief summary of our work.

II. RELATED WORK

PKEET in the random oracle model. Yang et al. [3] first proposed the notion of PKEET. In PKEET, every user has the capability to check whether two ciphertexts under different public keys are encryptions of the same message. Tang [4] pointed out that the scheme [3] lacks an authorization mechanism to specify who can perform equality test operations between their ciphertexts. To resolve this problem, they put forward an FG-PKEET scheme, in which two users cooperatively generate and issue a token to a server for authorizing it to perform the equality test. Then Tang [5] formulated the concept of all-or-nothing (AoN)-PKEET, which introduces the user-specified authorization mechanism into PKEET. Ma et al. [6] presented the notion of public key encryption with delegated equality test (PKE-DET), where only the delegated party is allowed to conduct ciphertext comparison operations. Huang et al. [7] brought forward a public key encryption with authorized equality test (PKE-AET) scheme, designing two types of warrants (i.e., the receiver's warrant and cipher-warrant). The receiver's warrant and cipher-warrant authorize the tester to conduct the equality test on all of the receiver's ciphertexts and some specific one, respectively. Nevertheless, their scheme was pointed out by Lee et al. [8] that there exists a concrete adaptive chosen ciphertext attack (CCA2) on it. Ma et al. [9] put forward a PKEET with flexible authorization (PKEET-FA) scheme, in which four authorization types are provided to protect users' data. Then Lin et al. [10] brought forward a more efficient PKEET-FA scheme without bilinear pairings. Lee et al. [11] designed a semi-generic approach to build PKEET schemes relying on

indistinguishability against adaptive chosen-ciphertext attacks (IND-CCA2) secure public key encryption (PKE) schemes and the computational Diffie-Hellman (CDH) assumption. Nevertheless, requiring the CDH assumption limits the generality of their approach. Lin et al. [12] proposed a generic approach to construct PKEET schemes without imposing any restriction on underlying cryptographic assumptions. Lee et al. [13] further gave a generic transformation to build PKEET schemes without employing IND-CCA2 secure PKE schemes. Their transformation method merely uses PKE with randomness extractability. Qu et al. [14] presented the first certificateless PKEET (CL-PKEET) scheme, which can cope with the certificate management problem in PKEET and the key escrow problem in IBEET. Ling et al. [15] put forward the first group public key encryption with equality test (G-PKEET) scheme, supporting the group granularity authorization and resisting the insider attack.³ Shen et al. [16] further presented a pairing-free group public key encryption with equality test, which has a more efficient test algorithm compared with the scheme [15]. Susilo et al. [17] proposed the notions of PKE with multi-ciphertext equality test (PKE-MET) and PKE with flexible multi-ciphertext equality test (PKE-FMET), both of which are practical and suitable for the scenario of multi-ciphertext equality test. Lin et al. [18] put forward a PKEET supporting flexible designated authorization (PKEET-FDA) scheme. In PKEET-FDA, the user can authorize multiple testers to test the ciphertexts and the message merely needs to be encrypted for one time regardless of the number of authorized testers.

PKEET in the standard model. Lee et al. [19] designed a generic method for building PKEET constructions in the standard model by employing a 2-level HIBE scheme with selective indistinguishability against chosen-plaintext attacks (IND-CPA) security and a one-time strong signature. Nevertheless, their PKEET constructions suffer from the low efficiency. Huang et al. [20] proposed a novel concept of PKE with filtered equality test (PKE-FET). In PKE-FET, only the messages chosen by receivers can be performed the equality test. Nonetheless, their construction was pointed out by Lee et al. [21] that it can

³Considering that the ciphertext can be generated publicly, the tester may exhaustively guess the target message by testing the generated ciphertexts with the target one when the message space is of polynomial size.

be broken by launching an adaptive chosen ciphertext attack. That is, it does not achieve so-called IND-CCA2 security in the standard model. Zhang et al. [22] brought forward an efficient PKEET scheme under the decisional bilinear Diffie-Hellman (DBDH) assumption in the standard model. Compared with the scheme [19], theirs has shorter ciphertexts and trapdoors, and is more efficient in the encryption, decryption and test algorithms. Subsequently, Zeng et al. [23] presented the first PKEET scheme in the standard model without pairings. Ma et al. [24] proposed a public key encryption with outsourced equality test for cloud-based IoT environments, which can resist the offline message recovery attack. They also gave a generic construction by employing a smooth projective hash function with the Lin-Hom property and an instantiation based on the SXDH assumption. Zhao et al. [25] introduced the notion of public key encryption with tester verifiable equality test (PKE-TVET), which supports to verify the message consistency in the test and decryption processes. They also put forward a PKE-TVET construction in the standard model. In addition, the studies [41], [42], [43], [44], [45], [46] explored how to design lattice-based PKEET and lattice-based CL-PKEET schemes in the standard model.

IBEET in the random oracle model. Ma [27] first introduced the notion of IBEET, simplifying the certificate management of PKEET, and defined OW-ID-CCA2 security for IBEET. However, owing to not defining IND-ID-CCA2 security, their security model is incomplete. Lee et al. [11] presented a semi-generic method for generating IBEET constructions. Moreover, they defined IND-ID-CCA2 security for their constructions. Nevertheless, the limitation of their method is that it must rely on the BDH assumption. Then Lin et al. [12] removed the limitation of method [11], and presented a more generic and efficient method for building IBEET constructions. Wu et al. [28] brought forward an IBEET scheme for the mobile cloud environment. Unfortunately, it merely achieves OW-ID-CCA2 security. The studies [29], [30] designed IBEET schemes for the smart city. Wu et al. [31] pointed out that the scheme [3] suffers from the insider attack. To settle this problem, they defined a novel security notion, called weak-IND-ID-CCA2, for their proposed scheme. Nonetheless, Lee et al. [32] presented an attack to demonstrate that the scheme [31] does not achieve the claimed security requirement. Ling et al. [33] introduced the notion of group ID-based encryption with equality test (G-IBEET) and designed a scheme against the insider attack. In G-IBEET, the group authorization significantly reduces the computation overhead of generating trapdoors and the storage overhead of storing them, while guaranteeing that only the users in the group have the capability to generate ciphertexts. Lin et al. [38] introduced the time-based authorization mechanism into IBEET to put forward the first identity-based encryption with equality test and datestamp-based authorization (IBEET-DBA) scheme. In IBEET-DBA, the validity of trapdoor can be controlled by the embedded datestamp. However, the authorization mechanism of IBEET-DBA is inexpressive and inflexible, due to merely expressing a single time interval in a trapdoor and supporting three kinds of authorization. Yang et al. [34] proposed an efficient lattice-based IBEET scheme from the learning with errors (LWE) problem, in which the hash value of message is embedded

in the trapdoor instead of the ciphertext to reduce the size of ciphertext. As a result, its encryption and decryption algorithms are relatively efficient.

IBEET in the standard model. Duong et al. [35] pointed out that the method [19] can yield the first instantiation of IBEET based on lattices in the standard model, but how to give a direct lattice-based IBEET scheme remains an open problem. To fill this gap, they presented the first direct IBEET scheme in the standard model under the LWE problem. They further extended their scheme to resist the insider attack. Nevertheless, their scheme only achieves chosen-plaintext attacks (CPA) security. Then Susilo et al. [47] proposed a novel lattice-based IBEET scheme with CCA2 security. Nguyen et al. [36] brought forward a lattice-based IBEET supporting flexible authorization (IBEET-FA) scheme, which is IND-ID-CPA and OW-ID-CPA secure. Moreover, they extended their scheme to achieve CCA2 security employing the BCHK's transformation [48] with the price of both of the ciphertext's and secret key's sizes increase. Asano et al. [37] presented the first adaptively and CCA secure lattice-based IBEET schemes in the standard model. They designed a semi-generic construction and used several lattice-based IBE constructions as the underlying constructions.

Attribute-based encryption with equality test (ABEET). El-tayieb et al. [49] presented a key-policy ABEET (KP-ABEET) scheme using access trees. It is proven secure under the DBDH assumption. However, its security is insufficient, due to only being CPA-type secure and not achieving one-wayness security. Cui et al. [50] proposed a ciphertext-policy ABEET (CP-ABEET) scheme by integrating ciphertext-policy attribute-based encryption (CP-ABE) and PKEET. It utilizes linear secret sharing schemes (LSSS) as the policy of the access structure and provides flexible authorization. Additionally, it achieves one-wayness against selective access structure and adaptive chosen-ciphertext attacks (OW-SAS-CCA), and indistinguishability against selective access structure and adaptive chosen-ciphertext attacks (IND-SAS-CCA) in the random oracle model. Cui et al. [51] brought forward a CP-ABEET scheme supporting outsourced decryption for devices with low computation capacity. Their scheme has an efficient decryption algorithm, requiring merely a constant number of computation operations. Besides, it is proven to be outsourced version of OW-SAS-CCA security and that of IND-SAS-CCA security in the random oracle model. Li et al. [52] designed a new CP-ABEET scheme by leveraging the chameleon hash function and dummy attribute. It supports large universe and ciphertext validity test. Moreover, it is OW-SAS-CCA and IND-SAS-CCA secure in the standard model. Asano et al. [53] put forward a generic construction of CCA secure ABEET employing the CPA-secure ABE with delegability and one-time signature with strong unforgeability. Adopting this generic method, the ABEET instantiations with favorable properties, such as adaptive security and standard assumptions, can be generated. Nonetheless, due to the need of invoking the encryption and decryption algorithms of the underlying ABE construction twice, coupled with additional calculations in its associated algorithms, its instantiations may suffer from low efficiency. Qu et al. [54] proposed a

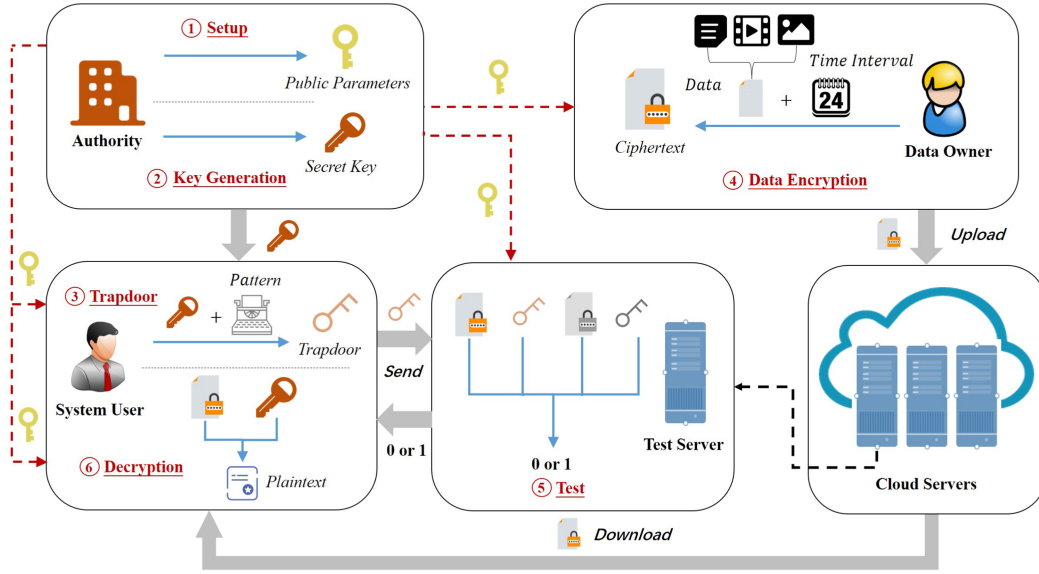


Fig. 2. The system architecture of IBEET-FTBA.

CP-ABEET scheme with traceability. Its tracing cost remains constant regardless of the number of users varying. Despite achieving adaptive security, their scheme is based on the composite order bilinear group, resulting in inefficiency and large parameter sizes.

A straightforward application of KP-ABEET may provide the similar functionality as our IBEET-FTBA in the multi-time-interval scenario. Nevertheless, in contrast to existing secure KP-ABEET schemes, our scheme is more compact and efficient. It eliminates redundant components that incur unnecessary computation and storage overhead, and does not rely on any strong cryptographic primitives (e.g., strongly unforgeable signatures) to achieve CCA security.

III. PRELIMINARY

A. Notations

For $n \in \mathbb{N}$, we define $[n] \stackrel{\text{def}}{=} \{1, 2, \dots, n\}$. For integers a, b and $a \leq b$, we define $[a, b] \stackrel{\text{def}}{=} \{a, a+1, \dots, b-1, b\}$. If A is an algorithm, $A \rightarrow a$ denotes that A is called and outputs a . A time interval T is a vector $(t_1, \dots, t_l) \in (\mathbb{Z}_p^*)^l$. A pattern P is a vector $(t'_1, \dots, t'_{l'}) \in (\mathbb{Z}_p^* \cup \{*\})^{l'}$, where $*$ is a wildcard. $T \approx P$ denotes the time interval T matches the pattern P , if and only if $l = l'$ and for all $i \in [l]$, $(t'_i = t_i) \vee (t'_i = *)$. We define $W(P)$ is the set containing all indices of the wildcard in P and $\overline{W}(P)$ is the set containing all indices of the non-wildcard in P , where $W(P) \cap \overline{W}(P) = \emptyset$ and $W(P) \cup \overline{W}(P) = [l']$.

B. Bilinear Maps

Definition 1 (Bilinear maps): Let $\mathbb{G}$ and $\mathbb{G}_T$ be two multiplicative cyclic groups of prime order p . Let g be a generator of $\mathbb{G}$ and e be a bilinear map, $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$. The bilinear map e has the following properties:

1. Bilinearity: for all $u, v \in \mathbb{G}$ and $a, b \in \mathbb{Z}_p$, we have $e(u^a, v^b) = e(u, v)^{ab}$.
2. Non-degeneracy: $e(g, g) \neq 1$, where 1 represents an identity element in $\mathbb{G}_T$.
3. Computable: There exists an efficient algorithm for the bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$.

The bilinear map e employed in our paper is symmetric. Namely, we have $e(u, v) = e(v, u)$.

C. Assumption

We use the truncated decision q -augmented bilinear Diffie-Hellman exponent (q -ABDHE) assumption proposed in [55], which is defined via a security game between a challenger and an attacker as follows.

Assumption 1 (truncated decision q -ABDHE assumption): The challenger calls the group generation algorithm with the security parameter λ as input, and picks two random group elements $g, g' \in \mathbb{G}$ and a random exponent $a \in \mathbb{Z}_p$. Then it sends to the attacker the group description $(p, \mathbb{G}, \mathbb{G}_T, e)$ and the following $q+3$ terms:

$$g', g'^{(a^{q+2})}, g, g^a, g^{(a^2)}, \dots, g^{(a^q)}.$$

Next, the challenger flips a random coin $b \in \{0, 1\}$. If $b = 0$, it sends the term $Z = Z_0 = e(g^{(a^{q+1})}, g') \in \mathbb{G}_T$ to the attacker. Otherwise, it sends a random term $Z = Z_1 = R \in \mathbb{G}_T$ to the attacker. Finally, the attacker outputs a guess $b' \in \{0, 1\}$ of b .

Definition 2: We say that the truncated decision q -ABDHE assumption holds if no probabilistic polynomial time (PPT) attacker $\mathcal{A}$ has a non-negligible advantage in the above security game. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}}^{q\text{-ABDHE}} = |\Pr[b' = b] - 1/2|$.

IV. DEFINITION AND SECURITY MODEL

In this section, we introduce the system architecture, definition and security model of our IBEET-FTBA.

A. System Architecture

The system architecture of IBEET-FTBA is shown in Fig. 2, which consists of the authority, data owners, recipients⁴ and the cloud server.

- **Authority.** The authority can manage identities, initialize the system by outputting the master key and the public parameters, authenticate users and issue secret keys for legitimate users.
- **Data owner.** Data owners can encrypt their data with target recipients' identities and time vectors recording when the ciphertexts are generated. Then they can upload these ciphertexts to the cloud server.
- **Recipient.** Firstly, recipients are able to send their identities to the authority and request secret keys. If they are legitimate users, they will obtain secret keys related to their identities. Secondly, recipients can authorize the cloud server as the tester to perform the equality test. Specifically, they generate trapdoors using their secret keys and pattern vectors specifying the ciphertexts generated during which time periods the cloud server can test. Then they send their trapdoors to the cloud server. Finally, recipients can try to decrypt ciphertexts with their own secret keys. If identities in ciphertexts are equal to those in secret keys, they will recover the plaintext successfully. Otherwise, the decryption will fail.
- **Cloud server.** The cloud server mainly has two functions in our system. The first one is to store ciphertexts uploaded by data owners. And the second one is to perform equality test operations and return results to corresponding users.

B. Definition

An IBEET-FTBA scheme consists of the following six algorithms:

- $\text{Setup}(1^\lambda, l) \rightarrow (PP, MSK)$: The setup algorithm takes as inputs a security parameter $\lambda \in N$ and a maximum depth l of the time interval tree. It outputs the public parameters PP and the master secret key MSK .
- $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$: The key generation algorithm takes as inputs the public parameters PP , the master secret key MSK and an identity ID . It outputs a secret key SK_{ID} related to the identity ID .
- $\text{Trapdoor}(PP, SK_{ID}, P) \rightarrow Td_{ID,P}$: The trapdoor generation algorithm takes as inputs the public parameters PP , a secret key SK_{ID} related with the identity ID and a pattern P . It outputs a trapdoor $Td_{ID,P}$ related to the pair (ID, P) .
- $\text{Encrypt}(PP, ID, T, m) \rightarrow CT_{ID,T}$: The encryption algorithm takes as inputs the public parameters PP , an

identity ID , a time interval T and a message m . It outputs a ciphertext $CT_{ID,T}$ associated with the pair (ID, T) .

- $\text{Test}(PP, CT_{ID'_A, T_A}^{(A)}, Td_{ID_A, P_A}^{(A)}, CT_{ID'_B, T_B}^{(B)}, Td_{ID_B, P_B}^{(B)}) \rightarrow 1/0$: The equality test algorithm takes as inputs the public parameters PP , two ciphertexts $CT_{ID'_A, T_A}^{(A)}, CT_{ID'_B, T_B}^{(B)}$ and two corresponding trapdoors $Td_{ID_A, P_A}^{(A)}, Td_{ID_B, P_B}^{(B)}$. If both of these two ciphertext/trapdoor pairs satisfy the following conditions, the algorithm outputs 1; otherwise, it outputs 0.
 - $ID_A = ID'_A$ and $ID_B = ID'_B$;
 - $T_A \approx P_A$ and $T_B \approx P_B$;
 - The message m_A in $CT_{ID'_A, T_A}^{(A)}$ is the same as the message m_B in $CT_{ID'_B, T_B}^{(B)}$.
- $\text{Decrypt}(PP, CT_{ID', T}, SK_{ID}) \rightarrow m$: The decryption algorithm takes as inputs the public parameters PP , a ciphertext $CT_{ID', T}$ associated with the pair (ID', T) and a secret key SK_{ID} related to the identity ID . It outputs m , if $ID = ID'$ and $CT_{ID', T}$ passes the validity check; otherwise, it outputs $\perp$.

Correctness: We say that an IBEET-FTBA scheme is *correct* if the following three conditions hold:

- 1) For any security parameter λ , any identity ID , any message m , any time interval T , it holds that

$$\Pr \left[\begin{array}{l} (PP, MSK) \leftarrow \text{Setup}(1^\lambda, l) \\ \mathcal{P} : SK_{ID} \leftarrow \text{KeyGen}(PP, MSK, ID) \\ CT_{ID, T} \leftarrow \text{Encrypt}(PP, ID, T, m) \end{array} \right] = 1,$$

where $\mathcal{P}$ denotes $\text{Decrypt}(PP, CT_{ID, T}, SK_{ID}) = m$.

- 2) For any security parameter λ , any identities ID_A, ID_B , any messages m_A, m_B , any time intervals T_A, T_B and any patterns P_A, P_B , it holds that

$$\Pr \left[\begin{array}{l} (PP, MSK) \leftarrow \text{Setup}(1^\lambda, l) \\ SK_{ID_A} \leftarrow \text{KeyGen}(PP, MSK, ID_A) \\ CT_{ID_A, T_A} \leftarrow \text{Encrypt}(PP, ID_A, T_A, m_A) \\ \mathcal{P}' : Td_{ID_A, P_A} \leftarrow \text{Trapdoor}(PP, SK_{ID_A}, P_A) \\ SK_{ID_B} \leftarrow \text{KeyGen}(PP, MSK, ID_B) \\ CT_{ID_B, T_B} \leftarrow \text{Encrypt}(PP, ID_B, T_B, m_B) \\ Td_{ID_B, P_B} \leftarrow \text{Trapdoor}(PP, SK_{ID_B}, P_B) \end{array} \right] = 1,$$

if $m_A = m_B$, $T_A \approx P_A$ and $T_B \approx P_B$, where $\mathcal{P}'$ denotes $\text{Test}(PP, CT_{ID_A, T_A}^{(A)}, Td_{ID_A, P_A}^{(A)}, CT_{ID_B, T_B}^{(B)}, Td_{ID_B, P_B}^{(B)}) = 1$.

- 3) For any security parameter λ , any identities ID_A, ID_B , any messages m_A, m_B , any time intervals T_A, T_B and any patterns P_A, P_B , it holds that

$$\Pr \left[\begin{array}{l} (PP, MSK) \leftarrow \text{Setup}(1^\lambda, l) \\ SK_{ID_A} \leftarrow \text{KeyGen}(PP, MSK, ID_A) \\ CT_{ID_A, T_A} \leftarrow \text{Encrypt}(PP, ID_A, T_A, m_A) \\ \mathcal{P}' : Td_{ID_A, P_A} \leftarrow \text{Trapdoor}(PP, SK_{ID_A}, P_A) \\ SK_{ID_B} \leftarrow \text{KeyGen}(PP, MSK, ID_B) \\ CT_{ID_B, T_B} \leftarrow \text{Encrypt}(PP, ID_B, T_B, m_B) \\ Td_{ID_B, P_B} \leftarrow \text{Trapdoor}(PP, SK_{ID_B}, P_B) \end{array} \right]$$

is negligible in λ for any two ciphertexts CT_{ID_A, T_A} and CT_{ID_B, T_B} such that $\text{Decrypt}(PP, CT_{ID_A, T_A}, SK_{ID_A}) \neq$

⁴Both of data owners and recipients are referred to as system users (users for short).

$\text{Decrypt}(PP, CT_{ID_B, T_B}, SK_{ID_B})$, where $\mathcal{P}'$ denotes $\text{Test}(PP, CT_{ID_A, T_A}^{(A)}, Td_{ID_A, P_A}^{(A)}, CT_{ID_B, T_B}^{(B)}, Td_{ID_B, P_B}^{(B)}) = 1$.

C. Security Model

- *Type-I adversaries*: This type of adversaries can acquire the trapdoor, which can test the challenge ciphertext. The intuition is that even if given any valid trapdoor, the adversary still can not recover the message in the challenge ciphertext.
- *Type-II adversaries*: This type of adversaries can not obtain the trapdoor related with the challenge pair. They act like the traditional adversary of IBE and are to determine which message is used to generate the challenge ciphertext.

Definition 3 (OW-ID-CCA2): An IBEET-FTBA scheme is one-wayness against adaptive identity and chosen-ciphertext attack (OW-ID-CCA2) secure against Type-I adversaries, if for any PPT adversary, it has at most a negligible advantage in λ in the following security game.

Setup: The challenger $\mathcal{B}$ runs $\text{Setup}(1^\lambda)$ and sends the public parameters PP to the adversary $\mathcal{A}$.

Phase 1: The adversary $\mathcal{A}$ issues the following types of queries, where each query is one of:

- *SecretKeyQuery*(ID): $\mathcal{A}$ may submit polynomially many secret key queries to $\mathcal{B}$. For each secret key query ID , $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$. Then $\mathcal{B}$ sends SK_{ID} to $\mathcal{A}$.
- *DecryptionQuery*($CT_{ID, T}$): $\mathcal{A}$ may submit polynomially many decryption queries to $\mathcal{B}$. For each decryption query $CT_{ID, T}$, $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$ and $\text{Decrypt}(PP, CT_{ID, T}, SK_{ID}) \rightarrow m$. Then $\mathcal{B}$ sends m to $\mathcal{A}$.
- *TrapdoorQuery*(ID, P): $\mathcal{A}$ may submit polynomially many trapdoor queries to $\mathcal{B}$. For each trapdoor query (ID, P) , $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$ and $\text{Trapdoor}(PP, SK_{ID}, P) \rightarrow Td_{ID, P}$. Then $\mathcal{B}$ sends $Td_{ID, P}$ to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ submits a challenge pair (ID^*, T^*) to the challenger $\mathcal{B}$, where the challenge identity ID^* has not been submitted as the input of *SecretKeyQuery* in *Phase 1*. Then $\mathcal{B}$ selects a random message m^* and calculates a challenge ciphertext by running $\text{Encrypt}(PP, ID^*, T^*, m^*) \rightarrow CT_{ID^*, T^*}^*$. Finally, $\mathcal{B}$ sends CT_{ID^*, T^*}^* to $\mathcal{A}$.

Phase 2: This basically the same as *Phase 1* but with the following constraints.

- *SecretKeyQuery*(ID): The constraint is $ID \neq ID^*$.
- *DecryptionQuery*($CT_{ID, T}$): The constraint is that any decryption query is not equal to CT_{ID^*, T^*}^* .

Guess: The adversary $\mathcal{A}$ outputs a guess m' of m^* . If $m' = m^*$, $\mathcal{A}$ wins the game. The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{OW-ID-CCA2}} = \Pr[m' = m^*].$$

Definition 4 (IND-ID-CCA2): An IBEET-FTBA scheme is indistinguishability against adaptive identity and chosen-ciphertext attack (IND-ID-CCA2) secure against Type-II adversaries, if for any PPT adversary, it has at most a negligible advantage in λ in the following security game.

Setup: The challenger $\mathcal{B}$ runs $\text{Setup}(1^\lambda)$ and sends the public parameters PP to the adversary $\mathcal{A}$.

Phase 1: The adversary $\mathcal{A}$ issues the following types of queries, where each query is one of:

- *SecretKeyQuery*(ID): $\mathcal{A}$ may submit polynomially many secret key queries to $\mathcal{B}$. For each secret key query ID , $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$. Then $\mathcal{B}$ sends SK_{ID} to $\mathcal{A}$.
- *DecryptionQuery*($CT_{ID, T}$): $\mathcal{A}$ may submit polynomially many decryption queries to $\mathcal{B}$. For each decryption query $CT_{ID, T}$, $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$ and $\text{Decrypt}(PP, CT_{ID, T}, SK_{ID}) \rightarrow m$. Then $\mathcal{B}$ sends m to $\mathcal{A}$.
- *TrapdoorQuery*(ID, P): $\mathcal{A}$ may submit polynomially many trapdoor queries to $\mathcal{B}$. For each trapdoor query (ID, P) , $\mathcal{B}$ runs $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$ and $\text{Trapdoor}(PP, SK_{ID}, P) \rightarrow Td_{ID, P}$. Then $\mathcal{B}$ sends $Td_{ID, P}$ to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ submits two equal length messages m_0, m_1 and a challenge pair (ID^*, T^*) to the challenger $\mathcal{B}$, where the challenge identity ID^* has not been submitted as the input of *SecretKeyQuery* and the challenge pair (ID^*, T^*) has not been satisfied by any input of *TrapdoorQuery* (i.e., $ID^* \neq ID$ or $T^* \not\approx P$) in *Phase 1*. Then $\mathcal{B}$ chooses a random bit $b \in \{0, 1\}$ and calculates a challenge ciphertext by running $\text{Encrypt}(PP, ID^*, T^*, m_b) \rightarrow CT_{ID^*, T^*}^*$. Finally, $\mathcal{B}$ sends CT_{ID^*, T^*}^* to $\mathcal{A}$.

Phase 2: This basically the same as *Phase 1* but with the following constraints.

- *SecretKeyQuery*(ID): The constraint is $ID \neq ID^*$.
- *DecryptionQuery*($CT_{ID, T}$): The constraint is that any decryption query is not equal to CT_{ID^*, T^*}^* .
- *TrapdoorQuery*(ID, P): The constraint is that the conditions (1) $ID = ID^*$ and (2) $T^* \approx P$ can not be satisfied simultaneously.

Guess: The adversary $\mathcal{A}$ outputs a guess b' of b . If $b' = b$, $\mathcal{A}$ wins the game. The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{IND-ID-CCA2}} = |\Pr[b' = b] - 1/2|.$$

V. OUR IBEET-FTBA SCHEME

A. Overview

Inspired by the idea of the HIBE scheme [56], we utilize the hierarchical tree to organize time periods of system. An example of hierarchical time interval tree is shown in Fig. 3, which consists of standard time intervals (e.g., year, day, hour) and self-defined time intervals (e.g., preceding and regular seasons). Under such circumstances, the form of any valid time interval in our system is a time vector, for instance, “October 21, 2023” can be represented as $T = (2023, \text{Season}, 10, 21)$. Then for the structure of ciphertext, a time vector is introduced to record the create time (or express the testable time period) of the ciphertext. For the structure of trapdoor, a pattern vector including wildcards, such as $P = (2023, *, *, 21)$, is embedded in to guarantee that the user can specify multiple time periods instead of a single

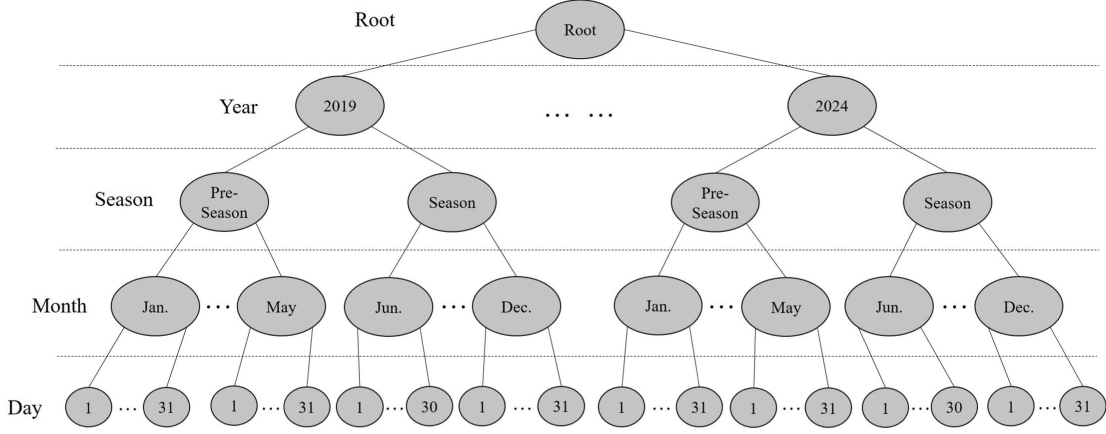


Fig. 3. An example of the time interval tree. **Root** is the root node of the tree, which does not represent any time interval. **Year**, **Month** and **Day** are all the standard time intervals, while **Season** is the self-defined time interval (including two values, namely, Pre-season and Season).

one at a time, which renders our authorization mechanism for equality test flexible. Concretely, any time vector in the set $\{(2023, \text{Pre-Season}, 1, 21), \dots, (2023, \text{Pre-Season}, 5, 21), (2023, \text{Season}, 6, 21), \dots, (2023, \text{Season}, 12, 21)\}$ can be derived from the pattern vector $P = (2023, *, *, 21)$. For simplicity, both the length j of the time interval T and the length k of the pattern P in our scheme are set to the maximum depth l of the time interval tree, a setting that aligns with real-world application scenarios. Moreover, ours can be also readily extended to support more situations of multiple time intervals, which can achieve further enhancement in expressiveness (refer to Section VII).

B. Construction

Our IBEET-FTBA construction works as follows.

- $\text{Setup}(1^\lambda, l) \rightarrow (PP, MSK)$: To generate the system parameters for an IBEET-FTBA with a maximum depth l of the time interval tree, the setup algorithm firstly calls the group generator algorithm $\mathcal{G}(1^\lambda)$ to obtain the descriptions of the groups and the bilinear map $D = (p, \mathbb{G}, \mathbb{G}_T, e)$. Then it randomly chooses $g, h, h', h'_2, h'_3, h_0, h_1, \dots, h_l \in \mathbb{G}$ and $\alpha, \alpha' \in \mathbb{Z}_p$. It computes $g_1 = g^\alpha, g'_1 = g^{\alpha'}, u_i = h_i^\alpha$ for each $i \in [0, l]$ and selects three cryptographic hash functions $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p, H_1 : \mathbb{G}_T \rightarrow \mathbb{G}$ and $H_2 : \mathbb{G}_T \rightarrow \{0, 1\}^{|\mathbb{G}|+|\mathbb{Z}_p|}$, where $|\mathbb{G}|$ and $|\mathbb{Z}_p|$ denote the lengths of an element in $\mathbb{G}$ and $\mathbb{Z}_p$, respectively. The public parameters and master secret key output are

$$PP = (H, H_1, H_2, g, g_1, g'_1, h, h', h'_2, h'_3, \{h_i, u_i\}_{i \in [0, l]}),$$

$$MSK = (\alpha, \alpha').$$

- $\text{KeyGen}(PP, MSK, ID) \rightarrow SK_{ID}$: To generate a secret key related with $ID \in \mathbb{Z}_p^*$, the key generation algorithm selects four random exponents $r, r', r'_2, r'_3 \in \mathbb{Z}_p$. Then it computes

$$K_1 = (hg^{-r})^{\frac{1}{\alpha-ID}}, K'_1 = (h'g^{-r'})^{\frac{1}{\alpha'-ID}},$$

$$K'_2 = (h'_2g^{-r'_2})^{\frac{1}{\alpha'-ID}}, K'_3 = (h'_3g^{-r'_3})^{\frac{1}{\alpha'-ID}}.$$

The secret key output is

$$SK_{ID} = (ID, r, r', r'_2, r'_3, K_1, K'_1, K'_2, K'_3).$$

- $\text{Trapdoor}(PP, SK_{ID}, P) \rightarrow Td_{ID, P}$: To generate a trapdoor related with the pair $(ID \in \mathbb{Z}_p^*, P = (t'_1, \dots, t'_k) \in (\mathbb{Z}_p^* \cup \{*\})^k)$, the trapdoor generation algorithm picks a random exponent $\mathfrak{t}$ and sets $t'_0 = ID$. Then it computes

$$D_1 = K_1 \left(\prod_{i \in \overline{W}(P) \cup \{0\}} (u_i h_i^{-ID})^{t'_i} \right)^{\mathfrak{t}}, D_2 = (g_1 g^{-ID})^{\mathfrak{t}},$$

$$\{D'_i = (u_i h_i^{-ID})^{\mathfrak{t}}\}_{i \in W(P)}.$$

The trapdoor Td_{ID} output is

$$Td_{ID, P} = ((ID, P), r, D_1, D_2, \{D'_i\}_{i \in W(P)}).$$

- $\text{Encrypt}(PP, ID, T, m) \rightarrow CT_{ID, T}$: To encrypt a message $m \in \mathbb{G}_T$ under the tuple $(ID \in \mathbb{Z}_p^*, T = (t_1, t_2, \dots, t_j) \in (\mathbb{Z}_p^*)^j)$, the encryption algorithm selects three random exponents s, s', z and sets $t_0 = ID$. Then it calculates

$$C_0 = g^z, C_1 = g_1^s g^{-sID}, C'_1 = g_1^{s'} g^{-s'ID}, C_2 = e(g, g)^s,$$

$$C'_2 = e(g, g)^{s'}, C_3 = m^z \cdot H_1(e(g, h)^s),$$

$$C_4 = \left(\prod_{i=0}^j (u_i h_i^{-ID})^{t_i} \right)^s,$$

$$C'_3 = (m||z) \oplus H_2(e(g, h')^{s'}), C'_4 = e(g, h'_2)^{s'} e(g, h'_3)^{s'w},$$

where $w = H(C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3)$. The ciphertext output is

$$CT_{ID, T} = ((ID, T), C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3, C'_4).$$

- $\text{Test}(PP, CT_{ID_A, T_A}^{(A)}, Td_{ID_A, P_A}^{(A)}, CT_{ID_B, T_B}^{(B)}, Td_{ID_B, P_B}^{(B)}) \rightarrow 1/0$: To compare whether $CT_{ID_A, T_A}^{(A)}$ and $CT_{ID_B, T_B}^{(B)}$ are encryptions of a same message, the test algorithm first checks whether the conditions ①–② can be satisfied simultaneously:

- ① $ID_A = ID'_A$ and $ID_B = ID'_B$;
 ② $T_A \approx P_A$ ($T_A = (t_{A,1}, \dots, t_{A,j_A})$, $P_A = (t'_{A,1}, \dots, t'_{A,k_A})$) and $T_B \approx P_B$ ($T_B = (t_{B,1}, \dots, t_{B,j_B})$, $P_B = (t'_{B,1}, \dots, t'_{B,k_B})$);

If not, the algorithm outputs $\perp$; otherwise, it computes

$$\begin{aligned}\bar{D}_{A,1} &= D_{A,1} \cdot \prod_{i \in W(P_A)} D_{A,i}^{t_{A,i}}, \\ B_A &= e(C_{A,1}, \bar{D}_{A,1}) \cdot C_{A,2}^{r_A} / e(C_{A,4}, D_{A,2}), \\ \Delta_A &= C_{A,3} / H_1(B_A).\end{aligned}$$

It can calculate Δ_B using the same method for computing Δ_A . Subsequently, it checks whether the following equation holds.

$$e(\Delta_A, C_{B,0}) \stackrel{?}{=} e(\Delta_B, C_{A,0}).$$

If so, the algorithm outputs 1 indicating $m_A = m_B$; otherwise, it outputs 0.

- Decrypt ($PP, CT_{ID,T}, SK_{ID}$) $\rightarrow m$: The decryption algorithm checks whether the conditions ①-② can be satisfied simultaneously:

- ① $ID = ID'$;
 ② $e(C'_1, K'_2(K'_3)^w) \cdot C_2^{r'_2 + r'_3 w} = C'_4$, where $w = H(C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3)$.

If not, the algorithm outputs $\perp$; otherwise, it computes

$$\begin{aligned}B &= e(C_1, K_1) \cdot C_2^r, \\ B' &= e(C'_1, K'_1) \cdot C_2^{r'}, \\ m||z &= C'_3 \oplus H_2(B').\end{aligned}$$

If the following two equations hold, the algorithm outputs m ; otherwise, it outputs $\perp$.

$$C_3 \stackrel{?}{=} m^z \cdot H_1(B), C_0 \stackrel{?}{=} g^z$$

— *Correctness*: We demonstrate the correctness of our construction.

- Test : If two ciphertexts $CT_{ID'_A, T_A}^{(A)}, CT_{ID'_B, T_B}^{(B)}$ and two trapdoors $Td_{ID_A, P_A}^{(A)}, Td_{ID_B, P_B}^{(B)}$ render the conditions ①-② hold, we have

$$\begin{aligned}\bar{D}_{A,1} &= D_{A,1} \cdot \prod_{i \in W(P_A)} D_{A,i}^{t_{A,i}} \\ &= (hg^{-r_A})^{\frac{1}{\alpha-ID_A}} \left(\prod_{i \in \bar{W}(P_A) \cup \{0\}} (u_i h_i^{-ID_A})^{t'_{A,i}} \right)^{t_A} \\ &\quad \cdot \prod_{i \in W(P_A)} ((u_i h_i^{-ID_A})^{t_A})^{t_{A,i}} \\ &= (hg^{-r_A})^{\frac{1}{\alpha-ID_A}} \left(\prod_{i=0}^{j_A} (u_i h_i^{-ID_A})^{t_{A,i}} \right)^{t_A}\end{aligned}$$

(Since $T_A \approx P_A$, $[j_A] = \bar{W}(P_A) \cup W(P_A)$ and $\forall i \in \bar{W}(P_A)$,

$$t'_{A,i} = t_{A,i}),$$

$$B_A = e(C_{A,1}, \bar{D}_{A,1}) \cdot C_{A,2}^{r_A} / e(C_{A,4}, D_{A,2})$$

$$\begin{aligned}&= e(g_1^{s_A} g^{-s_A ID_A}, (hg^{-r_A})^{\frac{1}{\alpha-ID_A}} \left(\prod_{i=0}^{j_A} (u_i h_i^{-ID_A})^{t_{A,i}} \right)^{t_A}) \\ &\quad \cdot (e(g, g)^{s_A})^{r_A} / e\left(\left(\prod_{i=0}^{j_A} (u_i h_i^{-ID_A})^{t_{A,i}}\right)^{s_A}, (g_1 g^{-ID_A})^{t_A}\right) \\ &= e(g_1 g^{-ID_A}, hg^{-r_A})^{\frac{s_A}{\alpha-ID_A}} \\ &\quad \cdot e(g_1 g^{-ID_A}, \prod_{i=0}^{j_A} (u_i h_i^{-ID_A})^{t_{A,i}})^{s_A t_A} \\ &\quad \cdot e(g, g)^{s_A r_A} / e\left(\prod_{i=0}^{j_A} (u_i h_i^{-ID_A})^{t_{A,i}}, g_1 g^{-ID_A}\right)^{s_A t_A} \\ &= e(g^{\alpha-ID_A}, hg^{-r_A})^{\frac{s_A}{\alpha-ID_A}} \cdot e(g, g)^{s_A r_A} \\ &= e(g, h)^{s_A} \cdot e(g, g)^{-s_A r_A} \cdot e(g, g)^{s_A r_A} \\ &= e(g, h)^{s_A}, \\ \Delta_A &= C_{A,3} / H_1(B_A) \\ &= m_A^{z_A} \cdot H_1(e(g, h)^{s_A}) / H_1(e(g, h)^{s_A}) \\ &= m_A^{z_A}.\end{aligned}$$

The equations $B_B = e(g, h)^{s_B}$ and $\Delta_B = m_B^{z_B}$ can be checked using the same method for the proofs of $B_A = e(g, h)^{s_A}$ and $\Delta_A = m_A^{z_A}$, respectively.

Then if $m_A = m_B$,

$$\begin{aligned}e(\Delta_A, C_{B,0}) &= e(m_A^{z_A}, g^{z_B}) = e(m_A, g)^{z_A z_B} \\ &= e(m_B, g)^{z_A z_B} = e(m_B^{z_B}, g^{z_A}) \\ &= e(\Delta_B, C_{A,0}).\end{aligned}$$

- Decrypt : If $ID = ID'$ and $CT_{ID', T}$ is a valid ciphertext, we have

$$\begin{aligned}&e(C'_1, K'_2(K'_3)^w) \cdot C_2^{r'_2 + r'_3 w} \\ &= e(g_1^{s'} g^{-s' ID}, (h'_2 g^{-r'_2})^{\frac{1}{\alpha-ID}} ((h'_3 g^{-r'_3})^{\frac{1}{\alpha-ID}})^w) \\ &\quad \cdot (e(g, g)^{s'})^{r'_2 + r'_3 w} \\ &= e(g^{s'(\alpha-ID)}, (h'_2 g^{-r'_2})^{\frac{1}{\alpha-ID}} (h'_3 g^{-r'_3})^{\frac{w}{\alpha-ID}}) \\ &\quad \cdot (e(g, g)^{s'})^{r'_2 + r'_3 w} \\ &= e(g^{s'(\alpha-ID)}, (g^{-r'_2} g^{-r'_3 w})^{\frac{1}{\alpha-ID}}) e(g^{s'(\alpha-ID)}, \\ &\quad (h'_2 (h'_3)^w)^{\frac{1}{\alpha-ID}}) \cdot (e(g, g)^{s'})^{r'_2 + r'_3 w} \\ &= (e(g, g)^{s'})^{-(r'_2 + r'_3 w)} \cdot e(g^{s'}, h'_2 (h'_3)^w) \cdot (e(g, g)^{s'})^{r'_2 + r'_3 w} \\ &= e(g, h'_2)^{s'} e(g, h'_3)^{s' w} \\ &= C'_4 \\ B &= e(C_1, K_1) \cdot C_2^r \\ &= e(g_1^{s_A} g^{-s_A ID}, (hg^{-r})^{\frac{1}{\alpha-ID}}) \cdot e(g, g)^{r s} \\ &= e(g, hg^{-r})^{\frac{s(\alpha-ID)}{\alpha-ID}} \cdot e(g, g)^{r s}\end{aligned}$$

$$\begin{aligned}
&= e(g, hg^{-r})^s \cdot e(g, g)^{rs} \\
&= e(g, h)^s \cdot e(g, g)^{-rs} \cdot e(g, g)^{rs} \\
&= e(g, h)^s,
\end{aligned}$$

$$\begin{aligned}
B' &= e(C'_1, K'_1) \cdot C'_2{}^{r'} \\
&= e(g_1^{s'} g^{-s'ID}, (h'g^{-r'})^{\frac{1}{\alpha'-ID}}) \cdot e(g, g)^{r's'} \\
&= e(g, h'g^{-r'})^{\frac{s'(\alpha'-ID)}{\alpha'-ID}} \cdot e(g, g)^{r's'} \\
&= e(g, h')^{s'} \cdot e(g, g)^{-r's'} \cdot e(g, g)^{r's'} \\
&= e(g, h')^{s'}, \\
C'_3 \oplus H_2(B') &= (m||z) \oplus H_2(e(g, h')^{s'}) \oplus H_2(e(g, h')^{s'}) \\
&= m||z,
\end{aligned}$$

And then,

$$C_3 = m^z \cdot H_1(e(g, h)^s) = m^z \cdot H_1(B).$$

C. Security Proof

In this part, we provide the security analysis for our IBEET-FTBA scheme.

Theorem 1: If the truncated decisional q -ABDHE assumption holds, H_1 is a one-way hash function, H, H_2 are collision-resistant hash functions, our proposed IBEET-FTBA scheme is OW-ID-CCA secure against Type-I PPT adversaries in the standard model.

Proof: Assume there exists a Type-I PPT adversary $\mathcal{A}$ who can break our IBEET-FTBA scheme with non-negligible probability $\mathcal{E}$. Using this adversary, we construct a simulator $\mathcal{B}$ who can attack the truncated decisional q -ABDHE assumption with non-negligible probability.

Setup: $\mathcal{B}$ firstly generates four random polynomials $f(x), f'_1(x), f'_2(x), f'_3(x) \in \mathbb{Z}[x]$ of degree q , and picks a random exponent $\tilde{\alpha} \in \mathbb{Z}_p$ and three cryptographic hash functions $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p, H_1 : \mathbb{G}_T \rightarrow \mathbb{G}, H_2 : \mathbb{G}_T \rightarrow \{0, 1\}^{|\mathbb{G}|+|\mathbb{Z}_p|}$. Then $\mathcal{B}$ sets $g'_1 = g^a$, and computes $g_1 = g^{\tilde{\alpha}}, h = g^{f(\tilde{\alpha})}, h' = g^{f'_1(a)}, h'_2 = g^{f'_2(a)}, h'_3 = g^{f'_3(a)}$ using the terms in the employed assumption. Next, $\mathcal{B}$ randomly picks $l + 1$ exponents $\mu_0, \dots, \mu_l$. $\mathcal{B}$ calculates

$$\{h_i = g^{\mu_i}, u_i = g_1^{\mu_i}\}_{i \in [0, l]}.$$

$\mathcal{B}$ sends the public parameters PP to $\mathcal{A}$ as

$$PP = (H, H_1, H_2, g, g_1, g'_1, h, h', h'_2, h'_3, \{h_i, u_i\}_{i \in [0, l]}).$$

Phase 1: The adversary $\mathcal{A}$ adaptively issues three types of queries.

– *SecretKeyQuery*(ID): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s secret key queries. For each secret key query ID , if $ID = a$, $\mathcal{B}$ uses a to solve the truncated decisional q -ABDHE assumption directly. Otherwise, let $F(x)$ and $F'_i(x)$ denote the $(q-1)$ -degree polynomials $(f(x) - f(ID))/(x - ID)$ and $(f'_i(x) - f'_i(ID))/(x - ID)$ respectively, where $i \in \{1, 2, 3\}$.

$\mathcal{B}$ sets $r = f(ID), r' = f'_1(ID), r'_2 = f'_2(ID), r'_3 = f'_3(ID)$, and computes

$$\begin{aligned}
K_1 &= (hg^{-r})^{\frac{1}{\alpha-ID}} = g^{\frac{f(\tilde{\alpha})-f(ID)}{\alpha-ID}} = g^{F(\tilde{\alpha})}, \\
K'_1 &= (h'g^{-r'})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_1(a)-f'_1(ID)}{a-ID}} = g^{F'_1(a)}, \\
K'_2 &= (h'_2g^{-r'_2})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_2(a)-f'_2(ID)}{a-ID}} = g^{F'_2(a)}, \\
K'_3 &= (h'_3g^{-r'_3})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_3(a)-f'_3(ID)}{a-ID}} = g^{F'_3(a)}.
\end{aligned}$$

Note that, the terms $g^{F'_1(a)}, g^{F'_2(a)}, g^{F'_3(a)}$ can be calculated using the terms in the truncated decisional q -ABDHE assumption. Meanwhile, the term $g^{F(\tilde{\alpha})}$ is also computable. Finally, $\mathcal{B}$ returns the secret key SK_{ID} to $\mathcal{A}$ as

$$SK_{ID} = (ID, r, r', r'_2, r'_3, K_1, K'_1, K'_2, K'_3).$$

– *TrapdoorQuery*(ID, P): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s trapdoor queries. For each trapdoor query (ID, P) , $\mathcal{B}$ generates a secret key SK_{ID} as the *SecretKeyQuery* phase does and then runs the Trapdoor algorithm to obtain a trapdoor $Td_{ID, P}$. Eventually, $\mathcal{B}$ returns the trapdoor $Td_{ID, P}$ to $\mathcal{A}$ as

$$Td_{ID, P} = ((ID, P), r, D_1, D_2, \{D'_i\}_{i \in W(P)}).$$

– *DecryptionQuery*($CT_{ID, T}$): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s decryption queries. For each decryption query $CT_{ID, T}$, $\mathcal{B}$ firstly generates SK_{ID} as the *SecretKeyQuery* phase does, and then runs the Decrypt algorithm with SK_{ID} as input. If $CT_{ID, T}$ does not pass the validity check, $\mathcal{B}$ returns $\perp$. Otherwise, $\mathcal{B}$ returns m to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ submits a challenge pair $(ID^*, T^* = (t_1^*, \dots, t_j^*))$ to the simulator $\mathcal{B}$, where the challenge identity ID^* has not been submitted as the input of *SecretKeyQuery* in Phase 1. Then $\mathcal{B}$ selects a random message m^* . If $ID^* = a$, $\mathcal{B}$ uses a to solve the truncated decisional q -ABDHE assumption immediately. Otherwise, $\mathcal{B}$ computes $SK_{ID^*} = (ID^*, r, r', r'_2, r'_3, K_1, K'_1, K'_2, K'_3)$ for the challenge identity ID^* as the *SecretKeyQuery* phase does. Next, $\mathcal{B}$ lets $\bar{f}(x) = x^{q+2}$ and $\bar{F}(x) = (\bar{f}(x) - \bar{f}(ID^*)) / (x - ID^*)$, which is a polynomial of degree $q+1$. $\mathcal{B}$ randomly picks exponents $z, s \in \mathbb{Z}_p$ and sets $t_0^* = ID^*$. Subsequently, $\mathcal{B}$ implicitly sets $s' = (\log_g g') \bar{F}(a)$ and calculates

$$C_0 = g^z, C_1 = g_1^s g^{-sID^*}, C_2 = e(g, g)^s,$$

$$\begin{aligned}
C'_1 &= g_1^{s'} g^{-s'ID^*} \\
&= g^{s'(\alpha'-ID^*)} \\
&= g^{\bar{F}(a)(a-ID^*)} \\
&= g^{\bar{f}(a)-\bar{f}(ID^*)},
\end{aligned}$$

$$\begin{aligned}
C'_2 &= e(g, g)^{s'} \\
&= e(g, g^{\bar{F}(a)}) \\
&= e(g, \prod_{i=0}^{q+1} g^{\bar{F}(a)^i})
\end{aligned}$$

$$\begin{aligned}
&= Z \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i a^i}), \\
C_3 &= m^{*z} \cdot H_1(e(g, h)^s) \\
&= m^{*z} \cdot H_1(e(g^s, h)), \\
C_4 &= (\prod_{i=0}^j (u_i h_i^{-ID^*})^{t_i^*})^s \\
&= \prod_{i=0}^j (g_1^s g^{-s ID^*})^{t_i^* \mu_i} = C_1^{\sum_{i=0}^j \mu_i t_i^*}, \\
C'_3 &= (m^* || z) \oplus H_2(e(g, h')^{s'}) \\
&= (m^* || z) \oplus H_2(e(C'_1, K'_1) \cdot C_2'^{r'}), \\
C'_4 &= e(g, h_2')^{s'} e(g, h_3')^{s'w} \\
&= e(C'_1, K'_2 (K'_3)^w) \cdot C_2'^{r'_2 + r'_3 w},
\end{aligned}$$

where $w = H(C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3)$ and $\bar{F}_i$ is the coefficient of x^i in $\bar{F}(x)$. Note that, $\bar{F}_{q+1} = 1$ and the terms $\bar{f}(a) - \bar{f}(ID^*)$, $\prod_{i=0}^q g^{\bar{F}_i a^i}$ are computable. Finally, $\mathcal{B}$ returns the challenge ciphertext CT_{ID^*, T^*} to $\mathcal{A}$ as

$$CT_{ID^*, T^*} = ((ID^*, T^*), C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3, C'_4).$$

Phase 2: This basically the same as *Phase 1* but with the following constraints.

- *SecretKeyQuery*(ID): The constraint is $ID \neq ID^*$.
- *DecryptionQuery*($CT_{ID, T}$): The constraint is that any decryption query is not equal to CT_{ID^*, T^*} .

Guess: The adversary $\mathcal{A}$ outputs a guess m' of m^* to $\mathcal{B}$. If $m' = m^*$, $\mathcal{B}$ will output 0 to indicate that $Z = e(g^{(a^{q+1})}, g') \in \mathbb{G}_T$. Otherwise, $\mathcal{B}$ will output 1 to indicate that $Z = R \in \mathbb{G}_T$.

Analysis of $\mathcal{B}$'s advantage: The simulator $\mathcal{B}$'s advantage $\text{Adv}_{\mathcal{B}}^{q\text{-ABDHE}}$ can be computed as follows.

$$\begin{aligned}
\text{Adv}_{\mathcal{B}}^{q\text{-ABDHE}} &= \Pr[Z = Z_0] \Pr[m^* = m' | Z = Z_0] \\
&\quad + \Pr[Z = Z_1] \Pr[m^* \neq m' | Z = Z_1] - \frac{1}{2} \\
&= \frac{1}{2} \cdot \text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{OW-ID-CCA2}} + \frac{1}{2} \cdot 1 - \frac{1}{2} \\
&= \frac{1}{2} \cdot \mathcal{E} + \frac{1}{2} \cdot 1 - \frac{1}{2} = \frac{\mathcal{E}}{2}
\end{aligned}$$

Note that, the above expressions hold since

- 1) $\Pr[Z = Z_0] = \Pr[Z = Z_1] = 1/2$;
- 2) $\mathcal{B}$ does not abort in the simulation;
- 3) When $Z = Z_0 = e(g^{(a^{q+1})}, g')$, $C'_2 = e(g^{(a^{q+1})}, g') \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i a^i}) = e(g', \prod_{i=0}^{q+1} g^{\bar{F}_i a^i}) = e(g', g^{\bar{F}(a)}) = e(g^{\log_g g'}, g^{\bar{F}(a)}) = e(g, g)^{(\log_g g') \bar{F}(a)} = e(g, g)^{s'}$.

Obviously, in this case, CT_{ID^*, T^*} is a valid encryption of m^* in our IBEET-FTBA scheme. Therefore, the simulation is the same as a real scheme from the view of $\mathcal{A}$, namely, $\Pr[m^* = m' | Z = Z_0] = \text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{OW-ID-CCA2}} = \mathcal{E}$;

- 4) When $Z = Z_1 = R$, all information about m^* is hidden by $R \in \mathbb{G}_T$. Since the total number of elements in $\mathbb{G}_T$ is extremely large, $\Pr[m^* \neq m' | Z = Z_1]$ is equal to 1.

This completes the proof of Theorem 1. $\square$

Theorem 2: If the truncated decisional q -ABDHE assumption holds, H_1 is a one-way hash function, H, H_2 are collision-resistant hash functions, our proposed IBEET-FTBA scheme is IND-ID-CCA secure against Type-II PPT adversaries in the standard model.

Proof 2: Assume there exists a Type-II PPT adversary $\mathcal{A}$ who can break our IBEET-FTBA scheme with non-negligible probability $\mathcal{E}$. Using this adversary, we construct a simulator $\mathcal{B}$ who can attack the truncated decisional q -ABDHE assumption with non-negligible probability.

Setup: $\mathcal{B}$ firstly generates four random polynomials $f(x), f'_1(x), f'_2(x), f'_3(x) \in \mathbb{Z}[x]$ of degree q , and picks a random exponent $\tilde{a} \in \mathbb{Z}_p$ and three cryptographic hash functions $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_1 : \mathbb{G}_T \rightarrow \mathbb{G}$, $H_2 : \mathbb{G}_T \rightarrow \{0, 1\}^{|\mathbb{G}| + |\mathbb{Z}_p|}$. Then $\mathcal{B}$ sets $g'_1 = g^{\tilde{a}}$, and computes $g_1 = g_1^{\tilde{a}}$, $h = g^{\tilde{f}(\tilde{a})}$, $h' = g^{f'_1(a)}$, $h'_2 = g^{f'_2(a)}$, $h'_3 = g^{f'_3(a)}$ using the terms in the employed assumption. Next, $\mathcal{B}$ randomly picks $l + 1$ exponents $\mu_0, \dots, \mu_l$. $\mathcal{B}$ calculates

$$\{h_i = g^{\mu_i}, u_i = g_1^{\mu_i}\}_{i \in [0, l]}.$$

$\mathcal{B}$ sends the public parameters PP to $\mathcal{A}$ as

$$PP = (H, H_1, H_2, g, g_1, g'_1, h, h', h'_2, h'_3, \{h_i, u_i\}_{i \in [0, l]}).$$

Phase 1: The adversary $\mathcal{A}$ adaptively issues three types of queries.

– *SecretKeyQuery*(ID): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s secret key queries. For each secret key query ID , if $ID = a$, $\mathcal{B}$ uses a to solve the truncated decisional q -ABDHE assumption directly. Otherwise, let $F(x)$ and $F'_i(x)$ denote the $(q-1)$ -degree polynomials $(f(x) - f(ID))/(x - ID)$ and $(f'_i(x) - f'_i(ID))/(x - ID)$ respectively, where $i \in \{1, 2, 3\}$. $\mathcal{B}$ sets $r = f(ID)$, $r' = f'_1(ID)$, $r'_2 = f'_2(ID)$, $r'_3 = f'_3(ID)$ and calculates

$$\begin{aligned}
K_1 &= (hg^{-r})^{\frac{1}{\alpha-ID}} = g^{\frac{f(\tilde{a}\alpha) - f(ID)}{\tilde{a}\alpha-ID}} = g^{F(\tilde{a}\alpha)}, \\
K'_1 &= (h'g^{-r'})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_1(a) - f'_1(ID)}{a-ID}} = g^{F'_1(a)}, \\
K'_2 &= (h'_2g^{-r'_2})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_2(a) - f'_2(ID)}{a-ID}} = g^{F'_2(a)}, \\
K'_3 &= (h'_3g^{-r'_3})^{\frac{1}{\alpha'-ID}} = g^{\frac{f'_3(a) - f'_3(ID)}{a-ID}} = g^{F'_3(a)}.
\end{aligned}$$

Note that, the terms $g^{F(\tilde{a}\alpha)}, g^{F'_1(a)}, g^{F'_2(a)}, g^{F'_3(a)}$ can be computed using the terms in the truncated decisional q -ABDHE assumption. Finally, $\mathcal{B}$ returns the secret key SK_{ID} to $\mathcal{A}$ as

$$SK_{ID} = (ID, r, r', r'_2, r'_3, K_1, K'_1, K'_2, K'_3).$$

– *TrapdoorQuery*(ID, P): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s trapdoor queries. For each trapdoor query $(ID, P = (t'_1, \dots, t'_k))$, if $ID = a$, $\mathcal{B}$ uses a to solve the truncated decisional q -ABDHE assumption immediately. Otherwise, $\mathcal{B}$ com-

puts a polynomial $o(x) \in \mathbb{Z}_p[x]$ of degree $q - 1$ such that

$$f(x) = o(x)(x - ID) + C,$$

where $C \in \mathbb{Z}_p$ is a non-zero constant. Note that, if $C = 0$, $\mathcal{B}$ aborts. Then $\mathcal{B}$ randomly selects an exponent $\bar{t} \in \mathbb{Z}_p^*$ and implicitly sets $t = \bar{t}/(\bar{\alpha}a - ID)$. Next, it sets $r = C$ and calculates

$$\begin{aligned} D_1 &= K_1 \left(\prod_{i \in \overline{W}(P) \cup \{0\}} (u_i h_i^{-ID})^{t'_i} \right)^{\bar{t}} \\ &= (h g^{-r})^{\frac{1}{\alpha - ID}} \cdot g^{\frac{\bar{t}}{\alpha - ID} \sum_{i \in \overline{W}(P) \cup \{0\}} \mu_i t'_i} \\ &= g^{\frac{f(\bar{\alpha}a) - C}{\bar{\alpha}a - ID}} \cdot g^{\frac{\bar{t}}{\alpha - ID} \sum_{i \in \overline{W}(P) \cup \{0\}} \mu_i t'_i} \\ &= g^{o(\bar{\alpha}a)} \cdot g^{\frac{\bar{t}}{\alpha - ID} \sum_{i \in \overline{W}(P) \cup \{0\}} \mu_i t'_i}, \\ D_2 &= (g_1 g^{-ID})^{\bar{t}} = (g^{\bar{\alpha}a - ID})^{\bar{t}} = g^{\bar{t}}, \\ D'_i &= (u_i h_i^{-ID})^{\bar{t}} = (h_i^{\bar{\alpha}a - ID})^{\bar{t}} = h_i^{\bar{t}}, i \in W(P). \end{aligned}$$

Note that, since the degree of the polynomial $o(x)$ is $q - 1$, the term $g^{o(\bar{\alpha}a)}$ is computable. Eventually, $\mathcal{B}$ returns the trapdoor $Td_{ID,P}$ to $\mathcal{A}$ as

$$Td_{ID,P} = ((ID, P), r, D_1, D_2, \{D'_i\}_{i \in W(P)}).$$

– *DecryptionQuery*($CT_{ID,T}$): The simulator $\mathcal{B}$ responds to $\mathcal{A}$'s decryption queries. For each decryption query $CT_{ID,T}$, $\mathcal{B}$ firstly generates SK_{ID} as the *SecretKeyQuery* phase does, and then runs the Decrypt algorithm with SK_{ID} as input. If $CT_{ID,T}$ does not pass the validity check, $\mathcal{B}$ returns $\perp$. Otherwise, $\mathcal{B}$ returns m to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ submits two equal length messages m_0, m_1 and a challenge pair $(ID^*, T^* = (t_1^*, \dots, t_j^*))$ to the simulator $\mathcal{B}$, where the challenge identity ID^* has not been submitted as the input of *SecretKeyQuery* and the challenge pair (ID^*, T^*) has not been satisfied by any input of *TrapdoorQuery* (i.e., $ID^* \neq ID$ or $T^* \not\approx P$) in *Phase 1*. If $ID^* = a$, $\mathcal{B}$ uses a to solve the truncated decisional q -ABDHE assumption directly. Otherwise, $\mathcal{B}$ flips a fair coin $b \in \{0, 1\}$ and computes $SK_{ID^*} = (ID^*, r, r', r'_2, r'_3, K_1, K'_1, K'_2, K'_3)$ for the challenge identity ID^* as the *SecretKeyQuery* phase does. Then $\mathcal{B}$ lets $\bar{f}(x) = x^{q+2}$ and $\bar{F}(x) = (\bar{f}(x) - \bar{f}(ID^*)) / (x - ID^*)$, which is a polynomial of degree $q + 1$. $\mathcal{B}$ randomly picks an exponent $z \in \mathbb{Z}_p$ and sets $t_0^* = ID^*$. Afterwards, $\mathcal{B}$ implicitly sets $s = (\log_g g') \bar{F}(\bar{\alpha}a)$, $s' = (\log_g g') \bar{F}(a)$ and computes

$$\begin{aligned} C_0 &= g^z, \\ C_1 &= g_1^s g^{-sID^*} \\ &= g^{s(\alpha - ID^*)} \\ &= g^{\bar{F}(\bar{\alpha}a)(\bar{\alpha}a - ID^*)} \\ &= g^{\bar{f}(\bar{\alpha}a) - \bar{f}(ID^*)}, \\ C'_1 &= g_1^{s'} g^{-s'ID^*} \\ &= g^{s'(\alpha' - ID^*)} \end{aligned}$$

$$\begin{aligned} &= g^{\bar{F}(a)(a - ID^*)} \\ &= g^{\bar{f}(a) - \bar{f}(ID^*)}, \\ C_2 &= e(g, g)^s \\ &= e(g', g^{\bar{F}(\bar{\alpha}a)}) \\ &= e(g', \prod_{i=0}^{q+1} g^{\bar{F}_i \bar{\alpha}^i \cdot a^i}) \\ &= Z^{\bar{\alpha}^{q+1}} \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i \bar{\alpha}^i \cdot a^i}), \\ C'_2 &= e(g, g)^{s'} \\ &= e(g', g^{\bar{F}(a)}) \\ &= e(g', \prod_{i=0}^{q+1} g^{\bar{F}_i a^i}) \\ &= Z \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i a^i}), \end{aligned}$$

$$\begin{aligned} C_3 &= m_b^z \cdot H_1(e(g, h)^s) \\ &= m_b^z \cdot H_1(e(C_1, K_1) \cdot C_2^r), \\ C_4 &= \left(\prod_{i=0}^j (u_i h_i^{-ID^*})^{t_i^*} \right)^s \\ &= \prod_{i=0}^j (g_1^s g^{-sID^*})^{t_i^* \mu_i} = C_1^{\sum_{i=0}^j \mu_i t_i^*}, \\ C'_3 &= (m_b || z) \oplus H_2(e(g, h')^{s'}) \\ &= (m_b || z) \oplus H_2(e(C_1, K'_1) \cdot C_2^{r'}), \\ C'_4 &= e(g, h'_2)^{s'} e(g, h'_3)^{s'w} \\ &= e(C_1, K'_2 (K'_3)^w) \cdot C_2^{r'_2 + r'_3 w}, \end{aligned}$$

where $w = H(C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3)$ and $\bar{F}_i$ is the coefficient of x^i in $\bar{F}(x)$. Note that, $\bar{F}_{q+1} = 1$ and the terms $\bar{f}(\bar{\alpha}a) - \bar{f}(ID^*)$, $\bar{f}(a) - \bar{f}(ID^*)$, $\prod_{i=0}^q g^{\bar{F}_i a^i}$, $\prod_{i=0}^q g^{\bar{F}_i \bar{\alpha}^i \cdot a^i}$ are computable. Finally, $\mathcal{B}$ returns the challenge ciphertext CT_{ID^*, T^*} to $\mathcal{A}$ as

$$CT_{ID^*, T^*} = ((ID^*, T^*), C_0, C_1, C'_1, C_2, C'_2, C_3, C_4, C'_3, C'_4).$$

Phase 2: This basically the same as *Phase 1* but with the following constraints.

- *SecretKeyQuery*(ID): The constraint is $ID \neq ID^*$.
- *DecryptionQuery*($CT_{ID,T}$): The constraint is that any decryption query is not equal to CT_{ID^*, T^*} .
- *TrapdoorQuery*(ID, P): The constraint is that the conditions (1) $ID = ID^*$ and (2) $T^* \approx P$ can not be satisfied simultaneously.

Guess: The adversary $\mathcal{A}$ outputs a guess b' of b to $\mathcal{B}$. If $b' = b$, $\mathcal{B}$ will output 0 to indicate that $Z = e(g^{(a^{q+1})}, g') \in \mathbb{G}_T$. Otherwise, $\mathcal{B}$ will output 1 to indicate that $Z = R \in \mathbb{G}_T$.

Analysis of $\mathcal{B}$'s advantage: The simulator $\mathcal{B}$'s advantage $\text{Adv}_{\mathcal{B}}^{q\text{-ABDHE}}$ can be computed as follows. Note that we use $\text{Abo}_{\mathcal{B}}$ to denote $\mathcal{B}$ aborts in the simulation.

$$\begin{aligned} \text{Adv}_{\mathcal{B}}^{q\text{-ABDHE}} &= \Pr[Z = Z_0] \Pr[b = b' | Z = Z_0] \\ &\quad + \Pr[Z = Z_1] \Pr[b \neq b' | Z = Z_1] - \frac{1}{2} \\ &= \frac{1}{2} \cdot \text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{IND-ID-CCA2}} (1 - \Pr[\text{Abo}_{\mathcal{B}}]) \\ &\quad + \frac{1}{2} \cdot \frac{1}{2} - \frac{1}{2} \\ &\geq \frac{1}{2} \cdot \left(\frac{1}{2} + \varepsilon \right) \left(1 - \frac{q}{p} \right) + \frac{1}{4} - \frac{1}{2} \\ &= \frac{2(p-q)\varepsilon - q}{4p} \end{aligned}$$

Note that, the above expressions hold since

- 1) $\Pr[Z = Z_0] = \Pr[Z = Z_1] = 1/2$;
- 2) When $Z = Z_0 = e(g^{(a^{q+1})}, g')$ and $\mathcal{B}$ does not abort, $C'_2 = e(g^{(a^{q+1})}, g') \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i a^i}) = e(g', \prod_{i=0}^{q+1} g^{\bar{F}_i a^i}) = e(g', g^{\bar{F}(a)}) = e(g^{\log_g g'}, g^{\bar{F}(a)}) = e(g, g)^{(\log_g g') \bar{F}(a)} = e(g, g)^{s'}$. Similarly, $C_2 = Z^{\tilde{a}^{q+1}} \cdot e(g', \prod_{i=0}^q g^{\bar{F}_i \tilde{a}^i a^i}) = e(g, g)^s$. Obviously, in this case, CT is a valid encryption of m_b in our IBEET-FTBA scheme. Therefore, the simulation is the same as a real scheme from the view of $\mathcal{A}$, namely, $\Pr[b = b' | Z = Z_0] = \text{Adv}_{\mathcal{A}, \text{IBEET-FTBA}}^{\text{IND-ID-CCA2}} = \frac{1}{2} + \varepsilon$;
- 3) $\mathcal{B}$ aborts if $C = 0$. Considering that $f(x)$ is a randomly distributed polynomial, the probability of the event that a random input ID makes $C = 0$ (namely, ID is a root of $f(x)$) is at most q/p . Hence, we have $\Pr[\text{Abo}_{\mathcal{B}}] \leq q/p$;
- 4) When $Z = Z_1 = R$, all information about m_b is hidden by $R \in \mathbb{G}_T$, so $\Pr[b \neq b' | Z = Z_1]$ is equal to $\frac{1}{2}$.

Therefore, we have the relation that

$$\varepsilon \leq \frac{4p \text{Adv}_{\mathcal{B}}^{q\text{-ABDHE}} + q}{2(p-q)}.$$

This completes the proof of Theorem 2. $\square$

VI. PERFORMANCE ANALYSIS

A. Theoretical Analysis

We provide a detailed comparison among IBEOT [27], IBEET-DBA [38] and our IBEET-FTBA on the theoretical cost. For the computation overhead, we primarily take the exponentiation operation E in $\mathbb{G}$ and the pairing operation P , which are time-consuming, into account. For the storage overhead, the length $|\mathbb{G}|$ of an element in $\mathbb{G}$, the length $|\mathbb{G}_T|$ of an element in $\mathbb{G}_T$ and the length $|\mathbb{Z}_p|$ of an element in $\mathbb{Z}_p$ are mainly considered. For comparison, we let $l = k = j = 3$ and $|W_I| = |W| = 0$ as IBEET-DBA merely achieves three kinds of authorization and does not support wildcards. Besides, we

let $|m| = |\mathbb{G}_T|$ since we assume that the message m in IBEET-DBA [38] is an element in $\mathbb{G}_T$.

As illustrated in Table II, IBEOT [27] has the best performance in all the algorithms and the most compact sizes for nearly all the components (except CT), since it does not support the time-based authorization mechanism. For the scenario of single time interval, our IBEET-FTBA costs 6 more exponentiation operations in $\mathbb{G}$ in KeyGen, 5 more exponentiation operations in $\mathbb{G}$ in Trapdoor, 10 more exponentiation operations in $\mathbb{G}$ and 4 more pairing operations in Encrypt, 2 more exponentiation operations in $\mathbb{G}$ in Test, 6 more exponentiation operations in $\mathbb{G}$ as well as 1 less pairing operation in Decrypt compared with IBEET-DBA [38]. In the meantime, ours spends $8|\mathbb{G}|$, $2|\mathbb{G}| + 4|\mathbb{Z}_p|$, $|\mathbb{Z}_p|$ and $3|\mathbb{G}| + 2|\mathbb{G}_T| + |\mathbb{Z}_p|$ more bit lengths than IBEET-DBA [38] does for $|PP|$, $|SK|$, $|Td|$ and $|CT|$, respectively. By contrast, for the scenario of multiple time intervals, our IBEET-FTBA is far superior to IBEET-DBA [38] in Trapdoor and $|Td|$. Suppose the number N_{ti} of time intervals is 12, then 12 invocations of Trapdoor of IBEET-DBA [38] will be performed, while Trapdoor of ours will be only executed once. In this situation, ours costs 50 less exponentiation operations in $\mathbb{G}$ in the trapdoor generation phase and requires $21|\mathbb{G}| - |\mathbb{Z}_p|$ less bit lengths in storing trapdoors in comparison with IBEET-DBA [38]. Namely, Our IBEET-FTBA reduces the computation overhead by approximately 83.33% and the storage overhead by roughly 85.42%. Furthermore, the advantage of IBEET-FTBA will become more apparent with N_{ti} increasing. In light of the above analysis, we can draw a conclusion that our IBEET-FTBA scheme is more applicable to the situation of multiple time intervals, especially when N_{ti} is large.

B. Experimental Analysis

Experimental setup. We implement our construction along with the IBEET-DBA construction [38]⁵ as a benchmark in java Pairing-Based Cryptography Library (jPBC) [57] (version 2.0.0). We use the default Type A and Type D pairings⁶ from jPBC, where the former pairings are constructed on the elliptic curve $y^2 = x^3 + x$ over the field $\mathbb{F}_q$ for some suitable prime p ($|p| = 160$ bits) and the latter ones are constructed on some ordinary elliptic curve $y^2 = x^3 + ax + b$ over the field $\mathbb{F}_q$ for some suitable prime p ($|p| = 224$ bits). Then a series of experiments are carried out on a desktop PC with AMD Ryzen 7 8845HS (3.8 GHz * 8) and 32 GB RAM running Windows 11 Home 24H2 (version 26100.6584) and JDK 21.0.8. For each experiment, we sample 200 times and take the average execution time.

Execution time analysis. Table III shows the execution time of all the algorithms in IBEET-DBA [38] and our IBEET-FTBA under three types of authorization. For Setup, the execution time of IBEET-FTBA is longer than that of IBEET-DBA [38]. The average gaps between them are about 38.56 ms on the Type A pairings and 109.37 ms on the Type D pairings, which are acceptable since Setup is run by the key generation server and

⁵The source code has been published in GitHub, where the address is <https://github.com/cleverli2008/Snowman-J/tree/main/IBEET-FTBA>.

⁶The default Type A pairing offers a roughly 80-bit security level, while the default Type D pairing provides an approximately 100-bit security level.

TABLE II
COMPARISON AMONG IBEOT, IBEET-DBA AND OUR IBEET-FTBA ON THE THEORETICAL COST

Scheme	IBEOET [27]	IBEET-DBA [38]	Ours
KeyGen	$2E$	$2E$	$8E$
Trapdoor	—	$5E$	$(2k + 4)E$
Encrypt	$6E + 2P$	$7E + 2P$	$(2j + 11)E + 6P$
Test	$4P$	$(6 - 2j)E + 6P$	$(2 W_I + 2)E + 6P$
Decrypt	$2E + 2P$	$4P$	$6E + 3P$
$ PP $	$3 \mathbb{G} $	$7 \mathbb{G} $	$(2l + 9) \mathbb{G} $
$ SK $	$2 \mathbb{G} $	$2 \mathbb{G} $	$4 \mathbb{G} + 4 \mathbb{Z}_p $
$ Td $	$ \mathbb{G} $	$(5 - j) \mathbb{G} $	$(W + 2) \mathbb{G} + \mathbb{Z}_p $
$ CT $	$5 \mathbb{G} + \mathbb{Z}_p $	$3 \mathbb{G} + m $	$6 \mathbb{G} + 3 \mathbb{G}_T + \mathbb{Z}_p $

E : an exponentiation operation in $\mathbb{G}$, P : a pairing operation, k : the length of pattern vector, j : the length of time vector, $|SK|/|Td|/|CT|$: the size of secret key/trapdoor/ciphertext, $|\mathbb{G}|/|\mathbb{G}_T|/|\mathbb{Z}_p|$: the length of an element in $\mathbb{G}/\mathbb{G}_T/\mathbb{Z}_p$, $|m|$: the bit length of message m , $|W_I|$: the number of wildcards involved in the test algorithm, $|W|$: the number of wildcards in the pattern vector.

TABLE III
COMPARISON BETWEEN IBEET-DBA AND OUR IBEET-FTBA ON EXECUTION TIME OF VARIOUS ALGORITHMS

Pairing	Type	Scheme	Setup	KeyGen	Trapdoor	Encrypt	Test	Decrypt
Type A	year	IBEET-DBA [38]	61.99	41.20	24.46	87.66	45.75	26.14
		IBEET-FTBA	97.96	42.74	58.81	135.15	66.85	51.40
	month	IBEET-DBA [38]	61.15	41.81	24.87	86.92	44.83	25.63
		IBEET-FTBA	101.48	43.40	59.51	137.79	57.36	50.99
	day	IBEET-DBA [38]	62.16	41.37	25.66	86.71	43.65	25.62
		IBEET-FTBA	101.54	42.98	58.52	136.93	46.71	51.73
Type D	year	IBEET-DBA [38]	183.93	149.89	63.84	280.42	198.84	120.02
		IBEET-FTBA	294.97	142.21	155.85	419.32	290.68	203.40
	month	IBEET-DBA [38]	191.37	155.68	66.73	281.67	200.12	122.69
		IBEET-FTBA	298.51	143.16	156.85	415.14	255.67	195.09
	day	IBEET-DBA [38]	190.30	149.75	67.84	276.14	194.92	122.33
		IBEET-FTBA	300.24	145.24	158.38	419.74	225.53	196.67

The unit of execution time for each algorithm is millisecond (ms).

a one-time operation for the system. For KeyGen, our IBEET-FTBA is close to IBEET-DBA [38], where the average gaps are merely 1.58 ms on the Type A pairings and 8.24 ms on the Type D pairings. For Trapdoor, IBEET-DBA [38] is superior to ours in the single time interval setting (i.e., $N_{ti} = 1$). In contrast, our IBEET-FTBA is more efficient when $N_{ti} > 2$. For instance, if $N_{ti} = 5$, IBEET-FTBA is more than twice as fast as IBEET-DBA [38]. For Encrypt, our IBEET-FTBA spends about 49.53 ms on the Type A pairings and 138.66 ms on the Type D pairings more than IBEET-DBA [38] does. For Decrypt, ours spends about 25.58 ms on the Type A pairings and 76.71 ms on the Type D pairings more than IBEET-DBA [38] does. Considering that these two algorithms are usually run by user devices, they can be optimized by the online/offline encryption and outsourced decryption techniques respectively when devices' resources are limited. Note that, the execution time of the algorithms mentioned above in IBEET-DBA [38] and IBEET-FTBA is all basically constant, regardless of the authorization type varying. In contrast, for Test, both of IBEET-DBA [38] and IBEET-FTBA's execution time decrease with the authorization type changing from "year-type" to "day-type". The gaps between them are narrowed from 21.10 ms to 3.06 ms

on the Type A pairings and 91.84 ms to 30.61 ms on the Type D pairings. Owing to being run by the cloud server, such increases in execution time are acceptable.

Figs. 4 and 5 display the execution time of Setup, Trapdoor, Encrypt and Test in our IBEET-FTBA with different input parameters on the Type A pairings and Type D pairings, respectively.⁷ Except for the experiments in Figs. 4(a) and 5(a), the maximum depths l of time interval tree in other experiments are all set to 20.

Figs. 4(a) and 5(a) show the changing of Setup's execution time along with l varying from 5 to 50 with 5 as the step size. It increases linearly with the growth of l , and reaches about 374.86 ms in Fig. 4(a) and 1161.27 ms in Fig. 5(a) when $l = 50$. Figs. 4(b) and 5(b) illustrate the changing of Trapdoor's execution time along with the length k of pattern vector changing from 1 to 19 with 2 as the step size when the pattern vector consists of all wildcards (denoted by case I) and all non-wildcards (denoted by case II), respectively. Obviously, the execution time of the case I and case II in both Figs. 4(b)

⁷Considering that the execution time of KeyGen and Decrypt in IBEET-FTBA is constant, we do not analyze it here.

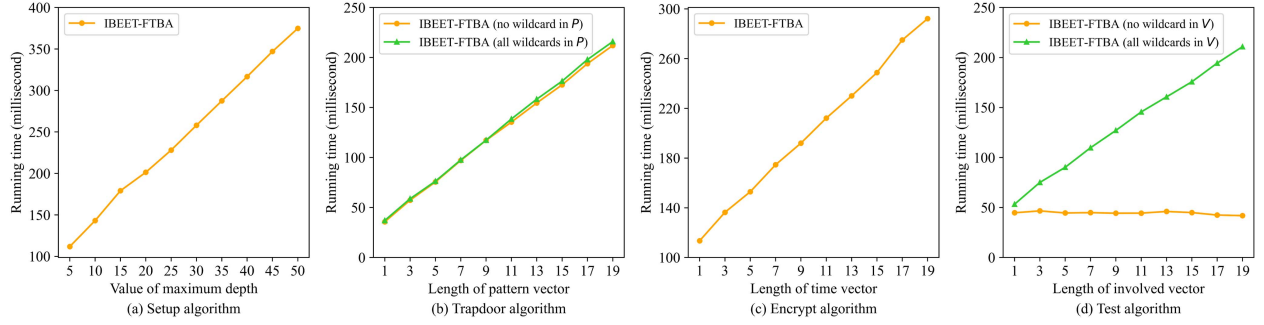


Fig. 4. Execution time analysis in various algorithms of our IBEET-FTBA under different maximum depths of the time interval tree and different lengths of vectors (based on the Type A pairings).

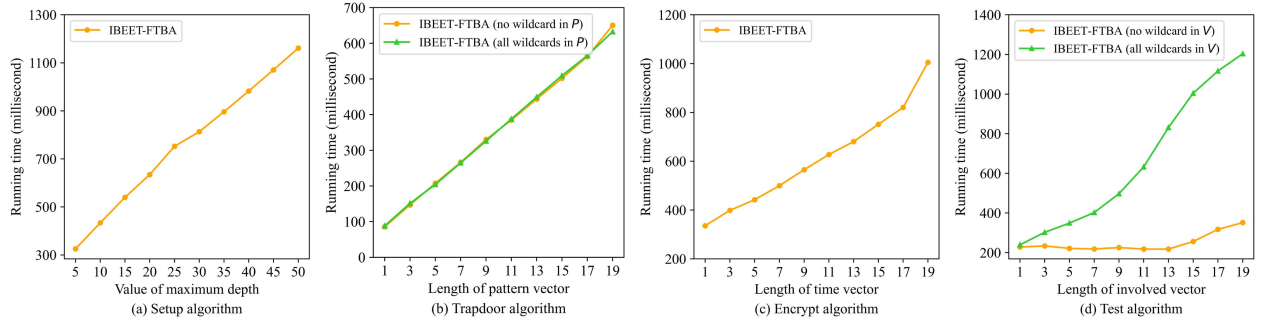


Fig. 5. Execution time analysis in various algorithms of our IBEET-FTBA under different maximum depths of the time interval tree and different lengths of vectors (based on the Type D pairings).

and 5(b) grows linearly with the increase of k . In addition, the execution time of case I is relatively close to that of case II under the same condition. Figs. 4(c) and 5(c) demonstrate the changing of Encrypt's execution time along with the length j of time vector varying from 1 to 19 with 2 as the step size. The execution time also increases with the growth of j . The average increments of execution time are around 19.86 ms in Fig. 4(c) and 74.39 ms in Fig. 5(c) per step. When $j = 19$, it reaches 292.19 ms in Fig. 4(c) and 1005.31 ms in Fig. 5(c). Figs. 4(d) and 5(d) display the changing of Test's execution time along with the length $|V|$ of involved vector changing from 1 to 19 with 2 as the step size when the involved vector consists of all wildcards (denoted by case i) and all non-wildcards (denoted by case ii), respectively. In case i , the execution time still increases with $|V|$, and the average increments are approximately 17.52 ms in Fig. 4(d) and 107.11 ms in Fig. 5(d) per step. By contrast, in case ii , the increase in execution time is relatively non-significant.

Storage overhead analysis. Fig. 6 illustrates the storage overhead of all components in IBEET-DBA [38] and our IBEET-FTBA under the "day-type" authorization. Here, the notation $|\cdot|$ denotes the size of a specific component. As observed, our IBEET-FTBA achieves a lower storage cost than IBEET-DBA [38] in terms of $|Td|$, whereas IBEET-DBA [38] is more efficient regarding $|PP|$, $|SK|$ and $|CT|$. Fig. 7 depicts the storage overhead of PP and Td in our IBEET-FTBA using the Type A and Type D pairings, respectively, with varying input parameters. Specifically, Fig. 7(a) and (b) demonstrate the variation of $|PP|$ as l increases from 2 to 10 with a step size of

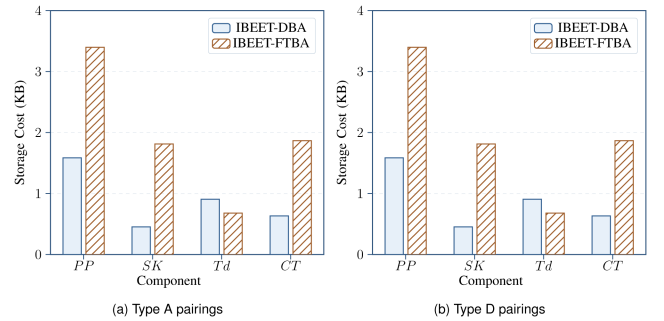


Fig. 6. Comparison between IBEET-DBA and our IBEET-FTBA on storage overhead of various components.

1. The overhead grows linearly with l , reaching approximately 6.73 KB in Fig. 7(a) and 7.2 KB in Fig. 7(b) when $l = 10$. Fig. 7(c) and (d) present the impact of k (ranging from 2 to 10) on $|Td|$. We consider two scenarios: a vector consisting of all wildcards (case 1) and all non-wildcards (case 2). The storage overhead for case 1 increases linearly with k , while that of case 2 remains constant.

In a nutshell, our IBEET-FTBA achieves the significant improvements in security and expressiveness without incurring massive losses on efficiency compared with IBEET-DBA [38]. Furthermore, ours is suitable for the scenario of multiple time periods, especially when the number of time periods is quite numerous.

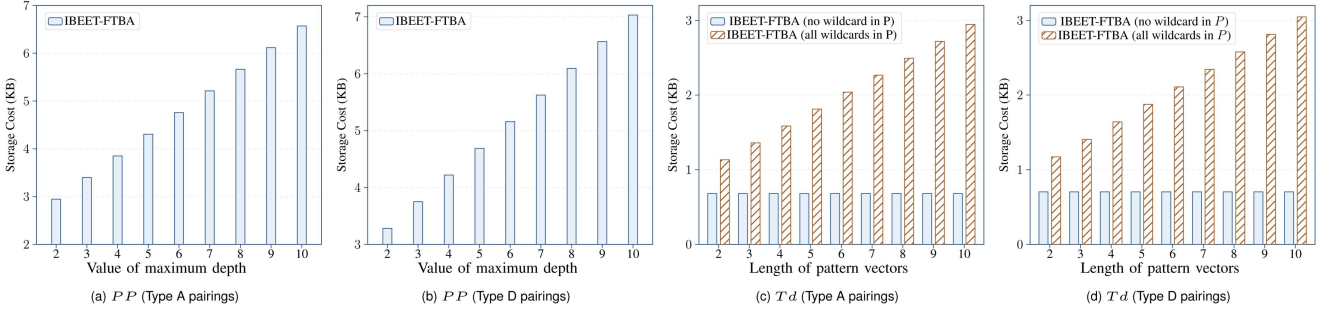


Fig. 7. Storage overhead analysis in various components of our IBEET-FTBA under different maximum depths of the time interval tree and different lengths of pattern vectors (based on the Type A and Type D pairings).

VII. DISCUSSION

We briefly discuss an extension of our IBEET-FTBA scheme. In reality, users may have more kinds of complicated requirements for authorization. For instance, they may authorize the server to test the ciphertexts generated on 8th of each month in 2021, 7th of each month in 2022 and from December 2023 to January 2024. Obviously, a single trapdoor in our IBEET-FTBA scheme can not express such time intervals directly. Fortunately, ours can be easily extended to be more expressive by supporting a set $\mathbb{P}$ of pattern vectors instead of a single pattern vector P . In this way, these time intervals can be expressed in a trapdoor with the set $\mathbb{P} = \{(2021, *, 8), (2022, *, 7), (2023, 12, *), (2024, 1, *)\}$. At the construction level, only a few alterations need to be made. Firstly, assume the size $|\mathbb{P}|$ of pattern vector set $\mathbb{P}$ is κ and $\mathbb{P} = \{P_1, P_2, \dots, P_\kappa\}$, where for each $\eta \in [\kappa]$, $P_\eta = (t'_{\eta,1}, \dots, t'_{\eta,k_\eta})$ and $t'_{\eta,0}$ is set to ID . Then the structure of trapdoor in our IBEET-FTBA is reused κ times. Next, a series of random exponents $\tau_1, \dots, \tau_\kappa$ are utilized to re-randomize the terms corresponding to each pattern vector in the input set $\mathbb{P}$, respectively. Eventually, the structure of extended trapdoor is presented and its form is $Td_{ID, \mathbb{P}} = ((ID, \mathbb{P}), r, \{D_{\eta,1}, D_{\eta,2}, \{D'_{\eta,i}\}_{i \in W(P_\eta)}\}_{\eta \in [\kappa]})$, where $D_{\eta,1} = K_1(\prod_{i \in \overline{W(P_\eta)} \cup \{0\}} (u_i h_i^{-ID})^{t'_{\eta,i}})^{\tau_\eta}$, $D_{\eta,2} = (g_1 g^{-ID})^{\tau_\eta}$, $\{D'_{\eta,i} = (u_i h_i^{-ID})^{\tau_\eta}\}_{i \in W(P_\eta)}$.

VIII. CONCLUSION

In this paper, we presented an IBEET with flexible time-based authorization scheme in the standard model, achieving wildcards and imposing no constraint on authorization types. Then we proved our scheme to be OW-ID-CCA2 security against Type-I adversaries and IND-ID-CCA2 security against Type-II adversaries under the truncated decision q -ABDHE assumption. Subsequently, we gave a theoretical analysis of our IBEET-FTBA and the main existing IBEET constructions. Moreover, we implemented our IBEET-FTBA construction along with the IBEET-DBA construction as a benchmark and conducted a series of experiments. Both results demonstrate that our scheme is more applicable to the scenario of multiple time intervals. Finally, we further discussed an extension of our IBEET-FTBA, which supports more complex situations of multiple time intervals.

We will explore the issues of how to enable the trapdoor to function within a predefined restricted lifecycle rather than operating permanently, how to reduce the scaling overhead of public parameters size and lower the computational costs of the trapdoor and encryption algorithms, and how to design a lattice-based IBEET-FTBA scheme in our future work.

ACKNOWLEDGMENT

We appreciate the anonymous reviewers for their efforts in reviewing this paper and helping us improve our work.

REFERENCES

- [1] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, ACM, 2009, pp. 169–178.
- [2] D. Boneh, G. D. Crescenzo, R. Ostrovsky, and G. Persiano, "Public key encryption with keyword search," in *Proc. 23rd Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2004, pp. 506–522.
- [3] G. Yang, C. H. Tan, Q. Huang, and D. S. Wong, "Probabilistic public key encryption with equality test," in *Proc. 2010 Cryptographers' Track RSA Conf.*, Springer, 2010, pp. 119–131.
- [4] Q. Tang, "Towards public key encryption scheme supporting equality test with fine-grained authorization," in *Proc. 16th Australas. Conf. Inf. Secur. Privacy*, Springer, 2011, pp. 389–406.
- [5] Q. Tang, "Public key encryption supporting plaintext equality test and user-specified authorization," *Secur. Commun. Netw.*, vol. 5, no. 12, pp. 1351–1362, 2012.
- [6] S. Ma, M. Zhang, Q. Huang, and B. Yang, "Public key encryption with delegated equality test in a multi-user setting," *Comput. J.*, vol. 58, no. 4, pp. 986–1002, 2015.
- [7] K. Huang, R. Tso, Y. Chen, S. M. M. Rahman, A. Almogren, and A. Alamri, "PKE-AET: Public key encryption with authorized equality test," *Comput. J.*, vol. 58, no. 10, pp. 2686–2697, 2015.
- [8] H. T. Lee, S. Ling, J. H. Seo, and H. Wang, "CCA2 attack and modification of Huang et al.'s public key encryption with authorized equality test," *Comput. J.*, vol. 59, no. 11, pp. 1689–1694, 2016.
- [9] S. Ma, Q. Huang, M. Zhang, and B. Yang, "Efficient public key encryption with equality test supporting flexible authorization," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 3, pp. 458–470, Mar. 2015.
- [10] X. J. Lin, L. Sun, H. Qu, and X. Zhang, "Public key encryption supporting equality test and flexible authorization without bilinear pairings," *Comput. Commun.*, vol. 170, pp. 190–199, 2021.
- [11] H. T. Lee, S. Ling, J. H. Seo, and H. Wang, "Semi-generic construction of public key encryption and identity-based encryption with equality test," *Inf. Sci.*, vol. 373, pp. 419–440, 2016.
- [12] X. J. Lin, L. Sun, and H. Qu, "Generic construction of public key encryption, identity-based encryption and signcryption with equality test," *Inf. Sci.*, vol. 453, pp. 111–126, 2018.
- [13] H. T. Lee, S. Ling, J. H. Seo, and H. Wang, "Public key encryption with equality test from generic assumptions in the random oracle model," *Inf. Sci.*, vol. 500, pp. 15–33, 2019.
- [14] H. Qu, Z. Yan, X. J. Lin, Q. Zhang, and L. Sun, "Certificateless public key encryption with equality test," *Inf. Sci.*, vol. 462, pp. 76–92, 2018.

- [15] Y. Ling, S. Ma, Q. Huang, X. Li, and Y. Ling, "Group public key encryption with equality test against offline message recovery attack," *Inf. Sci.*, vol. 510, pp. 16–32, 2020.
- [16] X. Shen, B. Wang, L. Wang, P. Duan, and B. Zhang, "Group public key encryption supporting equality test without bilinear pairings," *Inf. Sci.*, vol. 605, pp. 202–224, 2022.
- [17] W. Susilo, F. Guo, Z. Zhao, and G. Wu, "PKE-MET: Public-key encryption with multi-ciphertext equality test in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 10, no. 2, pp. 1476–1488, Apr.–Jun. 2022.
- [18] H. Lin, F. Gao, H. Zhang, Z. Jin, W. Li, and Q. Wen, "Public key encryption with equality test supporting flexible designated authorization in cloud storage," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1460–1470, Mar. 2022.
- [19] H. T. Lee, S. Ling, J. H. Seo, H. Wang, and T. Youn, "Public key encryption with equality test in the standard model," *Inf. Sci.*, vol. 516, pp. 89–108, 2020.
- [20] K. Huang, R. Tso, and Y. Chen, "Somewhat semantic secure public key encryption with filtered-equality-test in the standard model and its extension to searchable encryption," *J. Comput. Syst. Sci.*, vol. 89, pp. 400–409, 2017.
- [21] H. T. Lee, S. Ling, J. H. Seo, and H. Wang, "Insecurity of the public key encryption with filtered equality test proposed by huang et al.," *IACR Cryptol. ePrint Arch.*, Paper 2020/749, 2020. [Online]. Available: <https://eprint.iacr.org/2020/749>
- [22] K. Zhang, J. Chen, H. T. Lee, H. Qian, and H. Wang, "Efficient public key encryption with equality test in the standard model," *Theor. Comput. Sci.*, vol. 755, pp. 65–80, 2019.
- [23] M. Zeng, J. Chen, K. Zhang, and H. Qian, "Public key encryption with equality test via hash proof system," *Theor. Comput. Sci.*, vol. 795, pp. 20–35, 2019.
- [24] S. Ma, Y. Zhong, and Q. Huang, "Efficient public key encryption with outsourced equality test for cloud-based IoT environments," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3758–3772, 2022.
- [25] Z. Zhao, W. Susilo, B. Wang, and K. Zeng, "Public-key encryption with tester verifiable equality test for cloud computing," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3396–3406, Oct.–Dec. 2023.
- [26] Y. Wang, Q. Huang, H. Li, M. Xiao, S. Ma, and W. Susilo, "Private set intersection with authorization over outsourced encrypted datasets," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4050–4062, 2021.
- [27] S. Ma, "Identity-based encryption with outsourced equality test in cloud computing," *Inf. Sci.*, vol. 328, pp. 389–402, 2016.
- [28] L. Wu, Y. Zhang, K. R. Choo, and D. He, "Efficient and secure identity-based encryption scheme with equality test in cloud computing," *Future Gener. Comput. Syst.*, vol. 73, pp. 22–31, 2017.
- [29] L. Wu, Y. Zhang, K. R. Choo, and D. He, "Efficient identity-based encryption scheme with equality test in smart city," *IEEE Trans. Sustain. Comput.*, vol. 3, no. 1, pp. 44–55, Jan.–Mar. 2018.
- [30] Y. Ming and E. Wang, "Identity-based encryption with filtered equality test for smart city applications," *Sensors*, vol. 19, no. 14, 2019, Art. no. 3046.
- [31] T. Wu, S. Ma, Y. Mu, and S. Zeng, "ID-based encryption with equality test against insider attack," in *Proc. 22nd Australas. Conf. Inf. Secur. Privacy*, Springer, 2017, pp. 168–183.
- [32] H. T. Lee, H. Wang, and K. Zhang, "Security analysis and modification of ID-based encryption with equality test from ACISP 2017," in *Proc. 23rd Australas. Conf. One Inf. Secur. Privacy*, Springer, 2018, pp. 780–786.
- [33] Y. Ling, S. Ma, Q. Huang, X. Li, Y. Zhong, and Y. Ling, "Efficient group ID-based encryption with equality test against insider attack," *Comput. J.*, vol. 64, no. 4, pp. 661–674, 2021.
- [34] Z. Yang, D. He, L. Qu, and Q. Ye, "An efficient identity-based encryption with equality test in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2983–2992, Jul.–Sep. 2023.
- [35] D. H. Duong, H. Q. Le, P. S. Roy, and W. Susilo, "Lattice-based IBE with equality test in standard model," in *Proc. 13th Int. Conf. Provable Secur.*, Springer, 2019, pp. 19–40.
- [36] G. L. D. Nguyen, W. Susilo, D. H. Duong, H. Q. Le, and F. Guo, "Lattice-based IBE with equality test supporting flexible authorization in the standard model," in *Proc. 21st Int. Conf. Cryptol. India*, Springer, 2020, pp. 624–643.
- [37] K. Asano, K. Emura, and A. Takayasu, "More efficient adaptively secure lattice-based IBE with equality test in the standard model," in *Proc. 25th Int. Conf. Inf. Secur.*, Springer, 2022, pp. 75–83.
- [38] X. J. Lin, Q. Wang, L. Sun, and H. Qu, "Identity-based encryption with equality test and timestamp-based authorization mechanism," *Theor. Comput. Sci.*, vol. 861, pp. 117–132, 2021.
- [39] R. Canetti, O. Goldreich, and S. Halevi, "The random oracle methodology, revisited," *J. ACM*, vol. 51, no. 4, pp. 557–594, 2004.
- [40] S. Goldwasser and Y. T. Kalai, "On the (in)security of the fiat-shamir paradigm," in *Proc. 44th Symp. Foundations Comput. Sci.*, 2003, pp. 102–113.
- [41] D. H. Duong, K. Fukushima, S. Kiyomoto, P. S. Roy, and W. Susilo, "A lattice-based public key encryption with equality test in standard model," in *Proc. 24th Australas. Conf. Inf. Secur. Privacy*, Springer, 2019, pp. 138–155.
- [42] D. H. Duong, P. S. Roy, W. Susilo, K. Fukushima, S. Kiyomoto, and A. Sipasseuth, "Lattice-based public key encryption with equality test in standard model, revisited," 2020, *arXiv:2005.03178*.
- [43] P. S. Roy, D. H. Duong, W. Susilo, A. Sipasseuth, K. Fukushima, and S. Kiyomoto, "Lattice-based public-key encryption with equality test supporting flexible authorization in standard model," *Theor. Comput. Sci.*, vol. 929, pp. 124–139, 2022.
- [44] D. H. Duong, P. S. Roy, W. Susilo, K. Fukushima, S. Kiyomoto, and A. Sipasseuth, "Chosen-ciphertext lattice-based public key encryption with equality test in standard model," *Theor. Comput. Sci.*, vol. 905, pp. 31–53, 2022.
- [45] K. Xiao, X. Chen, J. Huang, H. Li, and Q. Huang, "A lattice-based public key encryption scheme with delegated equality test," *Comput. Stand. Interfaces*, vol. 87, 2024, Art. no. 103758.
- [46] D. H. Duong, W. Susilo, M. K. Bui, and T. X. Khuc, "A lattice-based certificateless public key encryption with equality test in standard model," in *Proc. 15th Int. Conf. Inf. Secur. Cryptol.*, Springer, 2019, pp. 50–65.
- [47] W. Susilo, D. H. Duong, and H. Q. Le, "Efficient post-quantum identity-based encryption with equality test," in *Proc. 26th IEEE Int. Conf. Parallel Distrib. Syst.*, 2020, pp. 633–640.
- [48] D. Boneh, R. Canetti, S. Halevi, and J. Katz, "Chosen-ciphertext security from identity-based encryption," *SIAM J. Comput.*, vol. 36, no. 5, pp. 1301–1328, 2007.
- [49] N. Eltayeb, R. Elhabob, A. Hassan, and F. Li, "Fine-grained attribute-based encryption scheme supporting equality test," in *Proc. 18th Int. Algorithms Archit. Parallel Process.*, Springer, 2018, pp. 220–233.
- [50] Y. Cui, Q. Huang, J. Huang, H. Li, and G. Yang, "Ciphertext-policy attribute-based encrypted data equality test and classification," *Comput. J.*, vol. 62, no. 8, pp. 1166–1177, 2019.
- [51] Y. Cui, Q. Huang, J. Huang, H. Li, and G. Yang, "Outsourced ciphertext-policy attribute-based encryption with equality test," in *Proc. 14th Int. Conf. Inf. Secur. Cryptol.*, Springer, 2018, pp. 448–467.
- [52] C. Li, Q. Shen, Z. Xie, X. Feng, Y. Fang, and Z. Wu, "Large universe CCA2 CP-ABE with equality and validity test in the standard model," *Comput. J.*, vol. 64, no. 4, pp. 509–533, 2021.
- [53] K. Asano, K. Emura, A. Takayasu, and Y. Watanabe, "A generic construction of CCA-secure attribute-based encryption with equality test," in *Proc. 16th Int. Conf. Provable Practical Secur.*, Springer, 2022, pp. 3–19.
- [54] Z. Qu, S. Kumari, M. S. Obaidat, B. A. Alzahrani, and H. Xiong, "Traceable attribute-based encryption with equality test for cloud enabled e-health system," *IEEE J. Biomed. Health Inform.*, vol. 28, no. 9, pp. 5033–5042, Sep. 2024.
- [55] C. Gentry, "Practical identity-based encryption without random oracles," in *Proc. 25th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 2006, pp. 445–464.
- [56] M. H. Au, J. K. Liu, T. H. Yuen, and D. S. Wong, "Practical hierarchical identity based encryption and signature schemes without random oracles," *IACR Cryptol. ePrint Arch.*, Paper 2006/368, 2006. [Online]. Available: <http://eprint.iacr.org/2006/368>
- [57] A. D. Caro and V. Iovino, "jPBC: Java pairing based cryptography," in *Proc. 16th IEEE Symp. Comput. Commun.*, 2011, pp. 850–855.



Cong Li received the PhD degree from Peking University, Beijing, China, in 2022. He was a postdoctoral fellow with the School of Computer Science, Peking University. He is currently an assistant professor with the School of Software and Microelectronics, Peking University. His research interests include applied cryptography, blockchain, cloud security, and privacy-preserving computing.



Xinyu Feng received the PhD degree from Peking University, Beijing, China, in 2024. He is currently the CTO of Zhonghe Tech (Xiong'an) Co., Ltd., Xiong'an, China. His research interests include applied cryptography, blockchain, and AI security.



Zhonghai Wu received the PhD degree from Zhejiang University, Hangzhou, China, in 1997. He was a postdoctor and senior research fellow with the Institute of Computer Science and Technology, Peking University, Beijing, China. Since then he has been involved both in research and application development of distributed system and multimedia applications. He is currently the executive president with the School of Software and Microelectronics, Peking University. His research interests include context-aware services, embedded software and system, cloud services and security, and open innovation.



Qingni Shen received the PhD degree from the Institute of Software Chinese Academy of Sciences, Beijing, China, in 2006. She is currently a professor with the School of Software and Microelectronics, Peking University, Beijing, China. Her research interests include operating system and virtualization security, privacy preserving in cloud and Big Data, trusted computing. She is a senior member of IEEE, ACM and CCF member. She is now serving for many international conferences, including TrustCom, SecureComm and ever as the chair of publicity committee or the member of program committee of ICICS in 2017, 2015, 2013, 2011, 2009 and 2007.